

强化学习开卷考试携带资料

MDP、Bellman 方程、DP、Dyna-Q、MC、TD、SARSA、Q-learning、DQN、REINFORCE、Actor-Critic、TRPO、PPO

重点：公式、推导、算法流程、伪代码、算法关系与对比

目录

1	一页总览：强化学习的主线	6
1.1	从问题到算法的递进	6
1.2	常见分类速查	6
2	马尔可夫决策过程（MDP）	7
2.1	MDP 定义	7
2.2	策略、轨迹与回报	8
2.3	价值函数	8
3	Bellman 方程	9
3.1	Bellman 期望方程	9
3.2	Bellman 最优化方程	9
3.3	Bellman 算子的收缩性	10
4	动态规划：已知模型时的规划方法	10
4.1	策略评估	10
4.2	策略改进	11
4.3	策略迭代	11
4.4	价值迭代	11
4.5	策略迭代与价值迭代对比	12
5	Monte Carlo：用完整回报学习	12
5.1	MC 预测	12
5.2	MC 控制	13
5.3	Off-policy MC 与重要性采样	14
5.4	MC 特点	14
6	TD：用一步样本做 Bellman backup	14
6.1	TD(0) 预测	14

6.2	MC 与 TD 对比	15
6.3	n -step TD	15
7	SARSA：同策略 TD 控制	16
7.1	单步 SARSA	16
7.2	多步 SARSA	16
7.3	SARSA 的特点	17
8	Q-learning：异策略 TD 控制	17
8.1	更新公式	17
8.2	SARSA 与 Q-learning 对比	18
9	Dyna-Q：真实经验加模型规划	18
9.1	模型与更新	18
9.2	Dyna-Q 的地位	19
10	DQN：用神经网络近似 Q 函数	19
10.1	普通 DQN	19
10.2	Double DQN	20
10.3	Dueling DQN	21
10.4	三种 DQN 对比	21
11	策略梯度基础	21
11.1	Baseline 与 Advantage	22
12	REINFORCE	23
12.1	更新公式	23
12.2	REINFORCE 特点	23
13	Actor-Critic	23
13.1	一步 Actor-Critic	24
13.2	多步优势与 GAE	24
13.3	Actor-Critic 与 REINFORCE 对比	25
14	TRPO：带信赖域的策略优化	25
14.1	重要性比率与替代目标	25
14.2	TRPO 优化问题	25
14.3	自然梯度近似	26

14.4 TRPO 特点	26
15 PPO：用裁剪目标近似信赖域	27
15.1 PPO-Clip 目标	27
15.2 完整损失	27
15.3 TRPO 与 PPO 对比	28
16 算法总对比表	28
17 伪代码汇总：考场速查版	29
17.1 动态规划	29
17.2 MC 与 TD	30
17.3 SARSA、Q-learning 与 Dyna-Q	31
17.4 DQN 系列	32
17.5 策略梯度与 Actor-Critic	32
17.6 TRPO 与 PPO	33
18 Python 语言伪代码：考试默写版	33
18.1 动态规划：策略评估、策略迭代、价值迭代	34
18.2 MC：蒙特卡洛预测与控制	35
18.3 TD(0)、SARSA、n-step SARSA、Q-learning	36
18.4 Dyna-Q	37
18.5 DQN、Double DQN、Dueling DQN	38
18.6 REINFORCE 与 Actor-Critic	38
18.7 TRPO 与 PPO	39
19 算法之间的递进关系	40
19.1 从 Bellman 到价值学习	40
19.2 从 TD 到控制算法	41
19.3 从 Q-learning 到 DQN 系列	41
19.4 从价值法到策略梯度	41
19.5 Dyna-Q 的位置	42
20 常考概念辨析	42
20.1 同策略与异策略	42
20.2 在线与离线	42
20.3 有模型与无模型	42

20.4 基于价值与基于策略	43
20.5 探索与利用	43
21 答题模板与易错点	43
21.1 答算法题的通用模板	43
21.2 常见易错点	43
22 公式速查表	44
23 最后串讲：考场叙述版	45
24 高频简答题预测与参考答案	46
24.1 题 1：什么是 MDP？写出 Bellman 期望方程和 Bellman 最优化方程，并说明它们在强化学习算法中的作用。	46
24.2 题 2：比较策略迭代和价值迭代。二者如何从 Bellman 方程递进而来？	46
24.3 题 3：比较 MC 和 TD。为什么说 TD 连接了 MC 与 DP？	47
24.4 题 4：比较 SARSA 和 Q-learning。为什么一个是同策略，一个是异策略？	47
24.5 题 5：从 Q-learning 到 DQN、Double DQN、Dueling DQN 的改进脉络是什么？	48
24.6 题 6：比较 REINFORCE 和 Actor-Critic。为什么要从纯策略梯度发展到 Actor-Critic？	48
24.7 题 7：比较 TRPO 和 PPO。PPO 是如何继承并简化 TRPO 思想的？	49
24.8 题 8：Dyna-Q 为什么说是 model-free 和 model-based 的结合？与 DP 和 Q-learning 分别有什么关系？	49
24.9 题 9：请从“有模型/无模型、同策略/异策略、在线/离线、基于价值/基于策略”的角度对主要算法分类。	50
24.10 题 10：请串讲强化学习算法从 DP 到 PPO 的发展脉络。	50
24.11 题 11：课程感悟题：学习强化学习后，你对人工智能或未来智能体研究有什么理解和启发？	51
24.12 题 12：课程感悟题：请结合本课程算法，谈谈你对“探索与利用”和“稳定训练”的理解。	51
24.13 题 13：什么是策略改进定理？它为什么能保证策略迭代逐步变好？	51
24.14 题 14：说明 off-policy MC 中重要性采样的作用，并比较普通重要性采样和加权重要性采样。	52
24.15 题 15：请解释 n -step 方法的意义，并说明它如何体现偏差和方差折中。	52
24.16 题 16：什么是优势函数？为什么策略梯度、A-C、TRPO、PPO 中常用优势函数而不是直接用回报？	53
24.17 题 17：DQN 为什么需要经验回放和目标网络？如果没有它们会出现什么问题？	53

24.18 题 18: 比较价值函数方法和策略梯度方法。什么情况下更适合用策略梯度? . . .	53
24.19 题 19: 解释 GAE 的思想。它与 TD、MC、优势估计有什么关系?	54
24.20 题 20: 请说明强化学习中的“探索与利用”问题, 并分别举出价值法和策略法中的 处理方式。	54
24.21 题 21: 如果考试要求你为一个任务选择算法, 应如何根据任务特点选择 DP、DQN 或 PPO?	55
24.22 题 22: 结合强化学习课程, 谈谈未来强化学习研究可能面临的挑战和发展方向。	55
24.23 题 23: 证明或说明 Bellman 最优算子为什么是收缩映射, 并解释这对价值迭代收 敛性的意义。	55
24.24 题 24: 深度强化学习中的 deadly triad 是什么? 请结合 Q-learning、DQN 和 Double DQN 说明其影响与缓解方法。	56
24.25 题 25: 解释 Q-learning 和 DQN 中的过估计偏差。Double DQN 为什么能缓解这 一问题?	56
24.26 题 26: 推导策略梯度定理的核心步骤, 并说明为什么 baseline 不改变梯度期望。	57
24.27 题 27: TRPO 和 PPO 都限制新旧策略差异。请从优化目标、约束方式和近似误 差角度深入比较二者。	58
24.28 题 28: 请从“目标值 target”的角度统一比较 MC、TD、SARSA、Q-learning 和 DQN。	58
24.29 题 29: 为什么 Q-learning 是异策略算法但通常不需要像 off-policy MC 那样显式 使用重要性采样?	59
24.30 题 30: Dueling DQN 为什么要写成 $Q = V + A - \bar{A}$? 如果直接写 $Q = V + A$ 会 有什么问题?	59
24.31 题 31: 请比较普通 DQN、Double DQN、Dueling DQN 三者分别解决了什么问 题, 是否可以组合使用?	60
24.32 题 32: Actor-Critic 中 critic 估计不准会对 actor 产生什么影响? 如何缓解? . . .	60
24.33 题 33: 为什么 PPO 可以对同一批数据训练多个 epoch, 而普通 on-policy 策略梯 度通常不适合无限重复使用旧数据?	60
24.34 题 34: 请解释“奖励稀疏”和“延迟奖励”为什么会使强化学习困难, 并说明 MC、 TD、n-step、GAE 如何处理信用分配问题。	61
24.35 题 35: 如果环境模型是错的, 基于模型方法会出现什么问题? Dyna-Q 如何在样 本效率和模型偏差之间折中?	61
24.36 题 36: 请从偏差-方差角度比较 MC、TD、REINFORCE、Actor-Critic、TRPO/PPO。	62
24.37 题 37: 请设计一个答题框架: 若老师要求“比较两个强化学习算法”, 应该从哪些 维度回答?	62

1 一页总览：强化学习的主线

强化学习（Reinforcement Learning, RL）研究智能体（agent）如何在环境中通过交互学习策略，使长期累计回报最大。最核心的数学对象是马尔可夫决策过程（MDP）；最核心的方程是 Bellman 方程；最核心的算法脉络是：

MDP $\Rightarrow$ Bellman 方程 $\Rightarrow$

有模型：动态规划、Dyna-Q

无模型价值法：MC、TD、SARSA、Q-learning、DQN

策略梯度法：REINFORCE、A-C、TRPO、PPO

1.1 从问题到算法的递进

1.

若已知环境转移概率和奖励模型，可以直接对 Bellman 方程做规划，得到动态规划（DP）：策略迭代和价值迭代。
2.

若不知道模型，但能采样完整 episode，可以用蒙特卡洛（MC）用真实回报估计价值：不 bootstrap，但方差大。
3.

若不知道模型，并希望边采样边更新，可以用时序差分（TD）：用一步真实奖励加下一状态估计值作为目标，bootstrap，方差较小但有偏。
4.

在控制问题中，用 TD 更新动作价值函数得到 SARSA 和 Q-learning。SARSA 学当前行为策略的价值，Q-learning 学贪心目标策略的价值。
5.

状态空间太大时，用神经网络近似 $Q(s, a)$ 得到 DQN。经验回放和目标网络稳定训练；Double DQN 减少过估计；Dueling DQN 分解状态价值和优势函数。
6.

若动作连续或直接优化策略更自然，则用策略梯度。REINFORCE 是最基础的 Monte Carlo 策略梯度；Actor-Critic 用 critic 降低方差；TRPO 用 KL 约束限制策略更新；PPO 用裁剪目标近似 trust region，工程上更简单。
7.

Dyna-Q 站在 model-free 和 model-based 中间：真实交互学习 Q ，同时学习一个模型，并用模型生成的模拟经验做规划更新。

1.2 常见分类速查

分类	含义	典型算法	考点区别
有模型	已知或学习环境模型 $p(s', r \mid s, a)$	DP、Dyna-Q	能用模型做规划，不一定每步都要真实交互。

分类	含义	典型算法	考点区别
无模型	不显式使用转移概率模型，只从样本更新	MC、TD、SARSA、Q-learning、DQN、REINFORCE、A-C、TRPO、PPO	样本效率较低，但不要 求知道环境方程。
同策略	行为策略和被评估/改进的目标策略相同	SARSA、REINFORCE、A-C、TRPO、PPO	学的是自己实际执行的策略，较保守。
异策略	行为策略和目标策略不同	Q-learning、DQN、Dyna-Q 的 Q-learning 部分、off-policy MC	可用探索策略采样，同时学习贪心或另一目标策略。
在线	与环境交互过程中即时更新	TD、SARSA、Q-learning、Dyna-Q、DQN	数据一来就更新，适合 continuing 或长 episode。
离线	用已有批数据或 episode 结束后更新	MC、TRPO/PPO 常用 batch、offline RL	注意：MC 的 episode-end 更新不是严格意义的 batch offline RL。
基于价值	先学习 V 或 Q ，再由价值诱导策略	DP、MC、TD、SARSA、Q-learning、DQN	离散动作中常用 $\arg \max_a Q(s, a)$ 。
基于策略	直接参数化并优化 $\pi_{\theta}(a s)$	REINFORCE、A-C、TRPO、PPO	适合连续动作、随机策略、可微策略优化。
Actor-Critic	actor 学策略，critic 学价值/优势	A-C、A2C/A3C、TRPO、PPO	介于价值法和策略法之间，是现代策略优化主流结构。
深度强化学 习	用深度网络近似价值、策略或模型	DQN、Double/Dueling DQN、深度 A-C、PPO	解决高维状态，但稳定性和样本效率是重点。

2 马尔可夫决策过程（MDP）

2.1 MDP 定义

一个有限 MDP 通常记为

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma),$$

其中：

- $\mathcal{S}$ ：状态空间；
- $\mathcal{A}$ ：动作空间，若动作集合随状态变化，可写为 $\mathcal{A}(s)$ ；

- $p(s', r \mid s, a)$: 转移和奖励的联合概率;
- $r(s, a)$ 或 $r(s, a, s')$: 期望即时奖励;
- $\gamma \in [0, 1]$: 折扣因子。

马尔可夫性表示未来只依赖当前状态和动作，与更早历史无关：

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_0, A_0, \dots, S_t = s, A_t = a) = p(s', r \mid s, a).$$

2.2 策略、轨迹与回报

策略 π 给出状态下选择动作的概率：

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

若 $\pi(a \mid s) \in \{0, 1\}$ ，称为确定性策略，可写为 $a = \pi(s)$ 。

折扣回报定义为

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

有限 episode 中，若终止时刻为 T ：

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

2.3 价值函数

状态价值函数：

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

动作价值函数：

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

二者关系：

$$\begin{aligned} v_{\pi}(s) &= \sum_a \pi(a \mid s) q_{\pi}(s, a), \\ q_{\pi}(s, a) &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]. \end{aligned}$$

最优价值函数：

$$v_*(s) = \max_{\pi} v_{\pi}(s), \quad q_*(s, a) = \max_{\pi} q_{\pi}(s, a).$$

由最优动作价值函数可得到最优贪心策略：

$$\pi_*(s) \in \arg \max_a q_*(s, a).$$

3 Bellman 方程

3.1 Bellman 期望方程

从回报递推式出发：

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

对固定策略 π ，状态价值为

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]. \end{aligned}$$

这就是 v_π 的 Bellman 期望方程。

动作价值函数的 Bellman 期望方程：

$$q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a') \right].$$

算子形式：

$$(T_\pi v)(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v(s')],$$

于是

$$v_\pi = T_\pi v_\pi.$$

3.2 Bellman 最优化方程

最优状态价值函数满足：

$$v_*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')].$$

最优动作价值函数满足：

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

由 v_* 得策略：

$$\pi_*(s) \in \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')].$$

由 q_* 得策略：

$$\pi_*(s) \in \arg \max_a q_*(s, a).$$

3.3 Bellman 算子的收缩性

对任意两个价值函数 u, v ，若 $\gamma < 1$ ，Bellman 期望算子和最优算子在无穷范数下都是 γ -收缩：

$$\|T_\pi u - T_\pi v\|_\infty \leq \gamma \|u - v\|_\infty,$$

$$\|T_* u - T_* v\|_\infty \leq \gamma \|u - v\|_\infty.$$

因此反复应用 Bellman backup 会收敛到唯一不动点：

$$v_{k+1} = T_\pi v_k \Rightarrow v_k \rightarrow v_\pi, \quad v_{k+1} = T_* v_k \Rightarrow v_k \rightarrow v_*.$$

这是 DP、TD、Q-learning 等算法背后的统一思想。

4 动态规划：已知模型时的规划方法

动态规划 (Dynamic Programming, DP) 假设环境模型 $p(s', r | s, a)$ 已知。它不依赖采样，而是对全状态空间做期望 backup。

4.1 策略评估

给定策略 π ，求 v_π 。迭代形式：

$$v_{k+1}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')].$$

动作价值版本：

$$q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_k(s', a') \right].$$

伪代码：迭代策略评估

1. 初始化 $V(s)$ ，终止状态通常设为 0。
2. 重复直到 $\Delta < \theta$ ：
3. 对每个状态 s ，保存 $v \leftarrow V(s)$ 。
4. 执行 Bellman 期望 backup：

$$V(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

5. 更新误差 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 。
6. 返回 $V \approx v_\pi$ 。

4.2 策略改进

若已有 v_π ，对每个状态取一步贪心：

$$\pi'(s) \in \arg \max_a q_\pi(s, a)$$

其中

$$q_\pi(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')].$$

策略改进定理：若对所有 s 都有

$$q_\pi(s, \pi'(s)) \geq v_\pi(s),$$

则

$$v_{\pi'}(s) \geq v_\pi(s), \quad \forall s.$$

直观理解：每一步都至少不比旧策略的一步选择差，递归展开后整体回报不差。

4.3 策略迭代

策略迭代（Policy Iteration）交替执行：

$$\text{策略评估} \quad \pi_k \Rightarrow v_{\pi_k},$$

$$\text{策略改进} \quad \pi_{k+1}(s) \in \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi_k}(s')].$$

直到策略稳定。

伪代码：策略迭代

1. 初始化任意策略 π 和价值函数 V 。
2. **策略评估**：用迭代策略评估求 $V \approx v_\pi$ 。
3. **策略改进**：令

$$\pi_{\text{old}}(s) \leftarrow \pi(s).$$

4. 对每个 s ：

$$\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

5. 若所有状态 $\pi(s) = \pi_{\text{old}}(s)$ ，停止；否则回到策略评估。

4.4 价值迭代

价值迭代（Value Iteration）把策略评估截断到一步，直接用 Bellman 最优 backup：

$$V_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')].$$

收敛后取贪心策略：

$$\pi(s) \in \arg \max_a \sum_{s',r} p(s',r \mid s,a)[r + \gamma V(s')].$$

伪代码：价值迭代

- 1. 初始化 $V(s)$ 。
- 2. 重复直到 $\Delta < \theta$ ：
- 3. 对每个状态 s ，保存 $v \leftarrow V(s)$ 。
- 4. 更新

$$V(s) \leftarrow \max_a \sum_{s',r} p(s',r \mid s,a)[r + \gamma V(s')].$$

- 5. $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 。
- 6. 输出贪心策略

$$\pi(s) \in \arg \max_a \sum_{s',r} p(s',r \mid s,a)[r + \gamma V(s')].$$

4.5 策略迭代与价值迭代对比

方面	策略迭代	价值迭代
核心思想	先较充分评估当前策略，再贪心改进	直接反复做最优 Bellman backup
更新式	$V \leftarrow T_\pi V$ ，再 $\pi \leftarrow \text{Greedy}(V)$	$V \leftarrow T_* V$
每轮成本	策略评估可能较贵	单轮较便宜
轮数	通常较少	通常较多
输出策略	每轮都有显式策略	最后从 V 提取策略
共同点	都需要完整模型和遍历状态动作空间，都是基于 Bellman 方程的动态规划。	

5 Monte Carlo：用完整回报学习

MC 方法不需要模型，只需要从环境中采样 episode。核心是用实际回报 G_t 作为价值的目标。

5.1 MC 预测

对固定策略 π ，状态价值估计：

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)].$$

动作价值估计：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)].$$

若用样本均值形式：

$$V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_i(s).$$

first-visit MC：一个 episode 中某状态第一次出现时才更新。every-visit MC：每次出现都更新。样本足够时二者都可收敛。

伪代码：First-Visit MC 预测

1. 输入策略 π 。
2. 初始化 $V(s)$ 、访问计数或回报列表。
3. 按策略 π 生成一个完整 episode：

$$S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T.$$

4. 从后往前计算 $G \leftarrow \gamma G + R_{t+1}$ 。
5. 若 S_t 是本 episode 中第一次出现，则更新

$$V(S_t) \leftarrow V(S_t) + \alpha [G - V(S_t)].$$

6. 重复采样 episode。

5.2 MC 控制

用动作价值 $Q(s, a)$ 做控制，因为不知道模型时无法由 $V(s)$ 直接比较动作。常用 ϵ -greedy 策略：

$$\pi(a | s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}, & a = \arg \max_b Q(s, b), \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{其他动作.} \end{cases}$$

伪代码：On-policy MC 控制

1. 初始化 $Q(s, a)$ ，令 π 为相对于 Q 的 ϵ -greedy 策略。
2. 用当前 π 生成一个完整 episode。
3. 对 episode 中每个 first-visit 的 (S_t, A_t) ，计算回报 G_t 。
4. 更新

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)].$$

5. 用新的 Q 改进策略： $\pi \leftarrow \epsilon$ -greedy(Q)。

5.3 Off-policy MC 与重要性采样

若行为策略为 b ，目标策略为 π ，可用重要性采样修正分布差异：

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}.$$

普通重要性采样：

$$V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} \rho_i G_i.$$

加权重要性采样：

$$V(s) = \frac{\sum_{i=1}^{N(s)} \rho_i G_i}{\sum_{i=1}^{N(s)} \rho_i}.$$

考点：普通重要性采样无偏但方差可能极大；加权重要性采样有偏但方差较小。

5.4 MC 特点

- 不需要模型；
- 不 bootstrap，目标 G_t 是完整真实回报；
- 只能在 episode 结束后知道完整 G_t ，不适合无终止 continuing 任务的原始形式；
- 估计通常无偏，但方差较大；
- 是 REINFORCE 的价值估计基础。

6 TD：用一步样本做 Bellman backup

TD (Temporal Difference) 结合 MC 和 DP：像 MC 一样从经验中学习，不需要模型；像 DP 一样 bootstrap，用当前估计值构造目标。

6.1 TD(0) 预测

固定策略 π 下，TD 目标：

$$Y_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1}).$$

TD 误差：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

更新：

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t.$$

与 Bellman 期望方程的关系：TD 用单个样本

$$R_{t+1} + \gamma V(S_{t+1})$$

近似期望

$$\mathbb{E}_{\pi}[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s].$$

伪代码：TD(0) 预测

- 1. 输入策略 π ，初始化 V 。
- 2. 对每个 episode，初始化状态 S 。
- 3. 若 S 非终止，按 π 选动作 A ，执行后得到 R, S' 。
- 4. 计算

$$\delta = R + \gamma V(S') - V(S).$$

- 5. 更新

$$V(S) \leftarrow V(S) + \alpha \delta.$$

- 6. 令 $S \leftarrow S'$ ，继续。

6.2 MC 与 TD 对比

方面	MC	TD
目标	完整回报 G_t	$R_{t+1} + \gamma V(S_{t+1})$
是否 bootstrap	否	是
更新时机	episode 结束后	每一步后即可
偏差/方差	通常无偏、高方差	有偏、低方差
适用任务	episodic 更自然	episodic 和 continuing 都适用
与 Bellman 方程关系	直接采样回报均值	随机近似 Bellman backup

6.3 n -step TD

n 步回报：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

更新：

$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_t^{(n)} - V(S_t) \right].$$

特殊情况：

$$n = 1 \Rightarrow \text{TD}(0), \quad n \rightarrow T - t \Rightarrow \text{MC}.$$

因此 n -step 方法连接了 TD 和 MC。

7 SARSA: 同策略 TD 控制

SARSA 名字来自一次更新用到的五元组:

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}).$$

它是同策略控制: 行为策略和要学习的目标策略相同, 通常都是 ϵ -greedy。

7.1 单步 SARSA

TD 目标:

$$Y_t^{\text{SARSA}} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}).$$

TD 误差:

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t).$$

更新:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \delta_t.$$

伪代码: 单步 SARSA

1. 初始化 $Q(s, a)$ 。
2. 对每个 episode, 初始化 S 。
3. 根据 Q 的 ϵ -greedy 策略选择 A 。
4. 对每一步: 执行 A , 得到 R, S' 。
5. 若 S' 非终止, 根据同一个 ϵ -greedy 策略选择 A' 。
6. 更新

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)].$$

7. 令 $S \leftarrow S', A \leftarrow A'$, 直到终止。

7.2 多步 SARSA

n 步 SARSA 回报:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q(S_{t+n}, A_{t+n}),$$

若 $t+n \geq T$, 则 bootstrap 项去掉:

$$G_t^{(n)} = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}.$$

更新:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t^{(n)} - Q(S_t, A_t)].$$

伪代码: n -step SARSA

1. 初始化 Q ，设步数 n 。
2. 对每个 episode，初始化并按 ϵ -greedy 得到 S_0, A_0 。
3. 对时间步 $t = 0, 1, 2, \dots$ ，执行 A_t ，得到 R_{t+1}, S_{t+1} 。
4. 若未终止，按当前策略选择 A_{t+1} 。
5. 令 $\tau = t - n + 1$ 。当 $\tau \geq 0$ 时，计算

$$G = \sum_{k=\tau}^{\min(\tau+n-1, T-1)} \gamma^{k-\tau} R_{k+1}.$$

6. 若 $\tau + n < T$ ，补上 bootstrap 项：

$$G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n}).$$

7. 更新

$$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha[G - Q(S_\tau, A_\tau)].$$

8. 当 $\tau = T - 1$ 时结束 episode。

7.3 SARSA 的特点

SARSA 目标中使用实际下一动作 A_{t+1} ，所以会把探索动作可能造成的风险也纳入价值估计。在带风险的路径规划中，SARSA 往往比 Q-learning 更保守。

8 Q-learning：异策略 TD 控制

Q-learning 直接学习最优动作价值函数 q_* ，行为策略可用于探索，目标策略是贪心策略。

8.1 更新公式

Q-learning 的 TD 目标：

$$Y_t^Q = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a').$$

TD 误差：

$$\delta_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t).$$

更新：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \delta_t.$$

它是 Bellman 最优化方程

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid s, a]$$

的随机近似。

伪代码：Q-learning

1. 初始化 $Q(s, a)$ 。
2. 对每个 episode, 初始化 S 。
3. 根据行为策略选择动作 A , 常用 ϵ -greedy。
4. 执行 A , 得到 R, S' 。
5. 更新

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right].$$

6. 令 $S \leftarrow S'$, 直到终止。
7. 最终策略为 $\pi(s) = \arg \max_a Q(s, a)$ 。

8.2 SARSA 与 Q-learning 对比

方面	SARSA	Q-learning
策略类型	同策略	异策略
目标	$R + \gamma Q(S', A')$	$R + \gamma \max_{a'} Q(S', a')$
下一动作	使用实际执行的 A'	使用贪心动作, 不一定实际执行
学习对象	当前探索策略的动作价值	最优贪心策略的动作价值
风险偏好	会考虑探索风险, 较保守	假设未来贪心, 较激进
收敛目标	若 $\epsilon \rightarrow 0$, 可到最优	满足条件时直接收敛到 q_*

9 Dyna-Q: 真实经验加模型规划

Dyna-Q 把 model-free Q-learning 和 model-based planning 结合起来。每次真实交互后:

1. 用真实样本更新 Q ;
2. 用真实样本更新模型 $\hat{p}, \hat{r}$;
3. 从已见过的 (s, a) 中抽样, 用模型产生模拟转移, 再做若干次 Q-learning 更新。

9.1 模型与更新

最简单的确定性模型可存储:

$$\text{Model}(s, a) = (\hat{r}, \hat{s}').$$

真实经验更新:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right].$$

模拟经验更新:

$$Q(\tilde{S}, \tilde{A}) \leftarrow Q(\tilde{S}, \tilde{A}) + \alpha \left[\tilde{R} + \gamma \max_{a'} Q(\tilde{S}', a') - Q(\tilde{S}, \tilde{A}) \right],$$

其中 $(\tilde{R}, \tilde{S}') = \text{Model}(\tilde{S}, \tilde{A})$ 。

伪代码：Dyna-Q

1. 初始化 $Q(s, a)$ 和模型 Model 。
2. 对每一步真实交互，当前状态为 S 。
3. 用 ϵ -greedy 从 Q 选择动作 A ，执行后得到 R, S' 。
4. **直接 RL 更新：**

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)].$$

5. **模型学习：** 令 $\text{Model}(S, A) \leftarrow (R, S')$ 。
6. **规划：** 重复 n 次：
7. 随机选择以前出现过的 $(\tilde{S}, \tilde{A})$ 。
8. 用模型得到 $(\tilde{R}, \tilde{S}') = \text{Model}(\tilde{S}, \tilde{A})$ 。
9. 做 Q-learning 模拟更新。
10. 令 $S \leftarrow S'$ ，继续。

9.2 Dyna-Q 的地位

- 与 Q-learning 相同：核心价值更新仍是 off-policy TD control。
- 与 DP 相似：会使用模型做 planning。
- 与纯 model-based 不同：模型是从经验中学习出来的，且可以只用简单表格模型。
- 主要优点：同等真实交互次数下，利用模拟更新提高样本效率。
- 主要风险：模型错误会传播到价值函数，非平稳环境中旧模型可能误导规划。

10 DQN：用神经网络近似 Q 函数

当状态空间很大或是图像输入时，表格 $Q(s, a)$ 不可行。DQN 用神经网络 $Q(s, a; \theta)$ 近似动作价值函数。

10.1 普通 DQN

普通 DQN 的目标来自 Q-learning：

$$y_i = r_i + \gamma \max_{a'} Q(s'_i, a'; \theta^-),$$

其中 θ^- 是目标网络参数。损失函数：

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(y - Q(s, a; \theta))^2].$$

梯度下降方向：

$$\nabla_{\theta} L(\theta) = \mathbb{E} [2(Q(s, a; \theta) - y) \nabla_{\theta} Q(s, a; \theta)].$$

两个稳定技巧：

- 经验回放 (experience replay)：把转移 (s, a, r, s', d) 存入 buffer $\mathcal{D}$ ，随机采样 minibatch，打破样本相关性并提高数据复用。
- 目标网络 (target network)：用较旧参数 θ^- 计算目标 y ，每隔 C 步复制 $\theta^- \leftarrow \theta$ 或软更新，降低移动目标导致的不稳定。

伪代码：DQN

1. 初始化在线网络 $Q(\cdot, \cdot; \theta)$ 、目标网络 $Q(\cdot, \cdot; \theta^-)$ ，令 $\theta^- = \theta$ 。
2. 初始化经验回放池 $\mathcal{D}$ 。
3. 对每一步，按 ϵ -greedy 从 $Q(s, \cdot; \theta)$ 选动作 a 。
4. 执行动作，得到 r, s', d ，存入 $\mathcal{D}$ 。
5. 从 $\mathcal{D}$ 采样 minibatch $(s_i, a_i, r_i, s'_i, d_i)$ 。
6. 计算目标

$$y_i = r_i + \gamma(1 - d_i) \max_{a'} Q(s'_i, a'; \theta^-).$$

7. 最小化

$$\frac{1}{B} \sum_i (y_i - Q(s_i, a_i; \theta))^2.$$

8. 每隔 C 步更新目标网络 $\theta^- \leftarrow \theta$ 。

10.2 Double DQN

普通 DQN 的 $\max$ 同时负责选择动作和评估动作，容易产生过估计。Double DQN 把选择和评估分开：

$$a_i^* = \arg \max_{a'} Q(s'_i, a'; \theta),$$

$$y_i^{\text{Double}} = r_i + \gamma(1 - d_i) Q(s'_i, a_i^*; \theta^-).$$

在线网络 θ 选动作，目标网络 θ^- 评估该动作。这样能减轻最大化噪声带来的过估计偏差。

伪代码：Double DQN 的目标计算

1. 从 replay buffer 采样 $(s_i, a_i, r_i, s'_i, d_i)$ 。
2. 在线网络选下一动作：

$$a_i^* = \arg \max_{a'} Q(s'_i, a'; \theta).$$

3. 目标网络评估该动作：

$$y_i = r_i + \gamma(1 - d_i)Q(s'_i, a_i^*; \theta^-).$$

4. 用 $(y_i - Q(s_i, a_i; \theta))^2$ 更新在线网络。

10.3 Dueling DQN

Dueling DQN 改变网络结构，把 Q 分解为状态价值 V 和优势函数 A ：

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right).$$

减去平均优势是为了解决不可辨识性：因为

$$Q(s, a) = V(s) + A(s, a)$$

中同时给 V 加常数、给 A 减常数并不改变 Q 。

Dueling 的意义：很多状态下动作选择不太影响结果，此时单独学习 $V(s)$ 更高效；优势分支只需学习动作之间的相对差异。

10.4 三种 DQN 对比

版本	核心改动	解决问题	目标
DQN	replay buffer + target network	深度网络训练不稳定、样本相关	$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$
Double DQN	在线网络选动作，目标网络估值	Q-learning 的过估计	$y = r + \gamma Q(s', a^*; \theta^-)$ $a^* = \arg \max_{a'} Q(s', a'; \theta)$
Dueling DQN	网络分成 $V(s)$ 和 $A(s, a)$	状态价值和动作优势混在一起难学	目标可用 DQN 或 Double DQN，结构不同

11 策略梯度基础

价值法间接得到策略；策略梯度法直接参数化策略：

$$\pi_\theta(a \mid s),$$

并最大化期望回报：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0].$$

一条轨迹

$$\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$$

的概率为

$$p_{\theta}(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) p(s_{t+1}, r_{t+1} | s_t, a_t).$$

环境转移不依赖 θ ，所以

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

使用 log-derivative trick:

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau).$$

于是

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) G(\tau) d\tau \\ &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G(\tau) \right]. \end{aligned}$$

用因果性可把整条轨迹回报换成从 t 开始的回报 G_t :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t \right].$$

11.1 Baseline 与 Advantage

对任意只依赖状态的 baseline $b(s)$:

$$\mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) b(s)] = 0.$$

证明:

$$\begin{aligned} \mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) b(s)] &= b(s) \sum_a \pi_{\theta}(a | s) \frac{\nabla_{\theta} \pi_{\theta}(a | s)}{\pi_{\theta}(a | s)} \\ &= b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) \\ &= b(s) \nabla_{\theta} 1 = 0. \end{aligned}$$

所以可用

$$G_t - b(S_t)$$

替代 G_t ，不改变期望梯度，但可降低方差。常取

$$b(S_t) = V^{\pi}(S_t).$$

优势函数:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

策略梯度常写为:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) A^{\pi}(S_t, A_t)].$$

12 REINFORCE

REINFORCE 是最基础的 Monte Carlo 策略梯度算法。它用完整回报 G_t 估计策略梯度。

12.1 更新公式

无 baseline:

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

带 baseline:

$$\theta \leftarrow \theta + \alpha (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

若 baseline 为价值网络 $V_w(s)$, 还可更新

$$w \leftarrow w - \beta \nabla_w (G_t - V_w(S_t))^2.$$

伪代码: REINFORCE

1. 初始化策略参数 θ 。
2. 用 π_{θ} 采样一个完整 episode。
3. 对每个时间步 t , 计算

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}.$$

4. 更新策略参数:

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

5. 若使用 baseline, 则把 G_t 换成 $G_t - b(S_t)$ 。

12.2 REINFORCE 特点

- 同策略: 采样和优化的是同一个 π_{θ} ;
- 无模型: 不需要 $p(s', r | s, a)$;
- Monte Carlo: 用完整回报, 不 bootstrap;
- 梯度估计无偏, 但方差大;
- episode 结束后才能更新, 样本效率较低。

13 Actor-Critic

Actor-Critic (A-C) 同时学习:

- Actor: 策略 $\pi_{\theta}(a | s)$;
- Critic: 价值函数 $V_w(s)$ 或 $Q_w(s, a)$, 用于评价 actor 的动作。

13.1 一步 Actor-Critic

用 TD 误差估计优势：

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t).$$

critic 更新：

$$w \leftarrow w + \beta \delta_t \nabla_w V_w(S_t).$$

actor 更新：

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

这里 δ_t 近似优势：

$$\delta_t \approx A^\pi(S_t, A_t).$$

因为

$$\mathbb{E}[\delta_t | S_t = s, A_t = a] = q_\pi(s, a) - v_\pi(s) = A^\pi(s, a)$$

当 $V_w \approx v_\pi$ 时成立。

伪代码：一步 Actor-Critic

1. 初始化 actor 参数 θ 和 critic 参数 w 。
2. 当前状态 S ，按 π_θ 采样动作 A 。
3. 执行后得到 R, S' 。
4. 计算 TD 误差：

$$\delta = R + \gamma V_w(S') - V_w(S).$$

5. 更新 critic：

$$w \leftarrow w + \beta \delta \nabla_w V_w(S).$$

6. 更新 actor：

$$\theta \leftarrow \theta + \alpha \delta \nabla_\theta \log \pi_\theta(A | S).$$

7. 令 $S \leftarrow S'$ ，继续。

13.2 多步优势与 GAE

n 步 actor-critic 可使用

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V_w(S_{t+n}),$$

$$\hat{A}_t = G_t^{(n)} - V_w(S_t).$$

广义优势估计（GAE）常用于 TRPO/PPO：

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t),$$

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.$$

其中 λ 控制偏差-方差折中： λ 越接近 1 越像 MC，越接近 0 越像一步 TD。

13.3 Actor-Critic 与 REINFORCE 对比

方面	REINFORCE	Actor-Critic
梯度权重	完整回报 G_t	TD 误差或优势估计 $\hat{A}_t$
是否 bootstrap	否	是
更新时机	episode 结束后	可每步或小批量更新
方差	大	较小
偏差	梯度估计通常无偏	critic 近似会引入偏差
结构	只有 actor，可加 baseline	actor + critic

14 TRPO：带信赖域的策略优化

TRPO (Trust Region Policy Optimization, 题目中有时误写为 TPRO) 解决策略梯度更新过大导致性能崩溃的问题。核心思想：最大化近似性能提升，同时限制新旧策略的 KL 距离。

14.1 重要性比率与替代目标

设旧策略为 $\pi_{\theta_{\text{old}}}$ ，新策略为 π_{θ} 。重要性比率：

$$r_t(\theta) = \frac{\pi_{\theta}(A_t \mid S_t)}{\pi_{\theta_{\text{old}}}(A_t \mid S_t)}.$$

一阶替代目标：

$$L_{\theta_{\text{old}}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right].$$

若 $\hat{A}_t > 0$ ，希望提高该动作概率；若 $\hat{A}_t < 0$ ，希望降低该动作概率。

14.2 TRPO 优化问题

TRPO 解如下约束优化：

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E}_t \left[\frac{\pi_{\theta}(A_t \mid S_t)}{\pi_{\theta_{\text{old}}}(A_t \mid S_t)} \hat{A}_t \right], \\ \text{s.t.} \quad & \mathbb{E}_t [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot \mid S_t) \parallel \pi_{\theta}(\cdot \mid S_t))] \leq \delta. \end{aligned}$$

其中 δ 是 trust region 半径。

14.3 自然梯度近似

把目标一阶近似，KL 二阶近似：

$$L(\theta_{\text{old}} + x) \approx L(\theta_{\text{old}}) + g^\top x,$$

$$\bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta_{\text{old}} + x) \approx \frac{1}{2} x^\top H x,$$

于是近似问题为

$$\max_x g^\top x, \quad \text{s.t.} \quad \frac{1}{2} x^\top H x \leq \delta.$$

解的方向是自然梯度：

$$x^* \propto H^{-1} g,$$

带尺度的解：

$$x^* = \sqrt{\frac{2\delta}{g^\top H^{-1} g}} H^{-1} g.$$

实际算法用共轭梯度近似求 $H^{-1}g$ ，再用 line search 保证 KL 约束和目标改善。

伪代码：TRPO

1. 用旧策略 $\pi_{\theta_{\text{old}}}$ 采样一批轨迹。
2. 用 critic 或 GAE 估计优势 $\hat{A}_t$ 。
3. 计算策略梯度 $g = \nabla_{\theta} L_{\theta_{\text{old}}}(\theta)|_{\theta=\theta_{\text{old}}}$ 。
4. 用共轭梯度求解 $Hx \approx g$ ，得到自然梯度方向。
5. 按

$$x \leftarrow \sqrt{\frac{2\delta}{x^\top H x}} x$$

调整步长。

6. 进行 backtracking line search，选择满足 KL 约束且提升目标的参数。
7. 更新 $\theta_{\text{old}} \leftarrow \theta$ 。

14.4 TRPO 特点

- 同策略、无模型、基于策略；
- 使用 KL 约束控制每次策略变化；
- 理论上更稳定，但实现复杂；
- 计算 Hessian-vector product、共轭梯度和线搜索，工程成本较高；
- 是 PPO 的直接前身。

15 PPO：用裁剪目标近似信赖域

PPO (Proximal Policy Optimization) 保留 TRPO “不要让新策略离旧策略太远” 的思想，但用简单的裁剪目标替代复杂的 KL 约束优化。

15.1 PPO-Clip 目标

重要性比率：

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

裁剪目标：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

分情况理解：

- 若 $\hat{A}_t > 0$ ，动作比平均好，想增大其概率，但 r_t 超过 $1 + \epsilon$ 后不再奖励。
- 若 $\hat{A}_t < 0$ ，动作比平均差，想降低其概率，但 r_t 低于 $1 - \epsilon$ 后不再奖励。
- 裁剪使策略更新不会因为少数样本过度放大。

15.2 完整损失

实践中常最大化：

$$L_t(\theta) = L_t^{\text{CLIP}}(\theta) - c_1 L_t^V(\theta) c_2 S[\pi_\theta](S_t),$$

其中价值损失为

$$L_t^V(\theta) = \left(V_\theta(S_t) - \hat{V}_t \right)^2,$$

熵奖励

$$S[\pi_\theta](s) = - \sum_a \pi_\theta(a | s) \log \pi_\theta(a | s)$$

鼓励探索。

伪代码：PPO-Clip

1. 初始化策略和值函数参数 θ 。
2. 用当前策略 $\pi_{\theta_{\text{old}}}$ 与环境交互，收集一批轨迹。
3. 计算回报目标 $\hat{V}_t$ 和优势 $\hat{A}_t$ ，常用 GAE。
4. 对同一批数据做 K 轮 minibatch 优化。
5. 每个 minibatch 中计算

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

6. 最大化

$$\mathbb{E}_t \left[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

同时最小化价值损失并加入熵奖励。

7. 完成多轮优化后，令 $\theta_{\text{old}} \leftarrow \theta$ ，重新采样。

15.3 TRPO 与 PPO 对比

方面	TRPO	PPO
控制更新幅度	显式 KL 约束	重要性比率裁剪, 可加 KL early stopping
优化方式	约束优化、共轭梯度、线搜索	普通 SGD/Adam minibatch
实现复杂度	高	低
稳定性	很稳定但成本高	稳定且工程友好
数据使用	一批数据通常更新较少	同一批数据可多 epoch 更新
共同点	都是同策略策略梯度方法，使用优势估计，目标是限制新旧策略差异。	

16 算法总对比表

算法	模型	策略属性	对象	核心目标/更新	关键特点
策略评估	有模型	固定策略	V^π	$V \leftarrow T_\pi V$	求当前策略价值。
策略迭代	有模型	贪心改进	V, π	评估 T_π ，改进 $\arg \max$	评估和改进交替。
价值迭代	有模型	最优控制	V^*	$V \leftarrow T_* V$	一步最优 backup。
Dyna-Q	学习模型	异策略	Q	真实和模拟 Q-learning	融合学习、模型、规划。
MC 预测	无模型	可同/异策略	V, Q	$X \leftarrow X + \alpha(G - X)$	完整回报，无 bootstrap。
TD(0)	无模型	固定策略	V^π	$V \leftarrow V + \alpha(R + \gamma V' - V)$	一步 bootstrap。
SARSA	无模型	同策略	Q^π	$R + \gamma Q(S', A')$	学实际探索策略。
n -step SARSA	无模型	同策略	Q^π	多步回报 $G_t^{(n)}$	连接单步 TD 和 MC。
Q-learning	无模型	异策略	Q^*	$R + \gamma \max_{a'} Q(S', a')$	学贪心最优策略。
DQN	无模型	异策略	Q_θ	target network 目标	深度 Q-learning。
Double DQN	无模型	异策略	Q_θ	在线选动作、目标网估值	减少过估计。

算法	模型	策略属性	对象	核心目标/更新	关键特点
Dueling DQN	无模型	异策略	V, A, Q	$Q = V + A - \bar{A}$	改网络结构。
REINFORCE	无模型	同策略	π_θ	$G_t \nabla \log \pi$	MC 策略梯度，高方差。
Actor-Critic	无模型	通常同策略	π_θ, V_w	$\delta \nabla \log \pi$	critic 降方差。
TRPO	无模型	同策略	π_θ, V_w	最大 surrogate, KL 约束	稳定但复杂。
PPO	无模型	同策略	π_θ, V_w	clipped surrogate	TRPO 的简化工程版。

17 伪代码汇总：考场速查版

前文每个算法小节下都有对应的“伪代码”块。本节把所有伪代码集中放在一起，考试时可直接翻本节。

17.1 动态规划

伪代码：迭代策略评估

- 1. 初始化 $V(s)$ ，给定策略 π ，阈值 θ 。
- 2. 重复：令 $\Delta \leftarrow 0$ 。
- 3. 对每个状态 s ，保存旧值 $v \leftarrow V(s)$ 。
- 4. 按 Bellman 期望方程更新：

$$V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')].$$

- 5. 更新 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 。
- 6. 若 $\Delta < \theta$ ，停止，返回 $V \approx v_\pi$ 。

伪代码：策略迭代

- 1. 初始化策略 π 和价值函数 V 。
- 2. 策略评估：反复执行

$$V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')]$$

直到近似收敛。

- 3. 策略改进：对每个 s 更新

$$\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')].$$

- 4. 若策略不再变化，停止；否则回到策略评估。

伪代码：价值迭代

- 1. 初始化 $V(s)$ ，阈值 θ 。

2. 重复：令 $\Delta \leftarrow 0$ 。
3. 对每个状态 s ，保存旧值 $v \leftarrow V(s)$ 。
4. 执行 Bellman 最优 backup：

$$V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

5. 更新 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 。
6. 收敛后输出

$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

17.2 MC 与 TD

伪代码：First-Visit MC 预测

1. 给定策略 π ，初始化 $V(s)$ 。
2. 用 π 采样完整 episode。
3. 从后往前计算 $G \leftarrow \gamma G + R_{t+1}$ 。
4. 若 S_t 是本 episode 中第一次出现，则更新

$$V(S_t) \leftarrow V(S_t) + \alpha [G - V(S_t)].$$

5. 重复采样 episode，直到估计稳定。

伪代码：On-policy MC 控制

1. 初始化 $Q(s, a)$ ，令 π 为相对 Q 的 ϵ -greedy 策略。
2. 用当前 π 采样完整 episode。
3. 对每个 first-visit 的 (S_t, A_t) 计算 G_t 。
4. 更新

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)].$$

5. 根据新 Q 更新 $\pi \leftarrow \epsilon$ -greedy(Q)。

伪代码：TD(0) 预测

1. 给定策略 π ，初始化 $V(s)$ 。
2. 每一步按 π 选动作并与环境交互，得到 R, S' 。
3. 计算 TD 误差：

$$\delta = R + \gamma V(S') - V(S).$$

4. 更新

$$V(S) \leftarrow V(S) + \alpha \delta.$$

5. 令 $S \leftarrow S'$ ，继续下一步。

17.3 SARSA、Q-learning 与 Dyna-Q

伪代码：单步 SARSA

1. 初始化 $Q(s, a)$ ，用 ϵ -greedy 选初始动作 A 。
2. 执行 A ，得到 R, S' 。
3. 在 S' 下仍按当前 ϵ -greedy 策略选 A' 。
4. 更新

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)].$$

5. 令 $S \leftarrow S'$ ， $A \leftarrow A'$ ，直到 episode 结束。

伪代码： n -step SARSA

1. 初始化 Q ，设置步数 n 。
2. 按 ϵ -greedy 生成序列 $S_0, A_0, R_1, S_1, A_1, \dots$ 。
3. 当 $\tau = t - n + 1 \geq 0$ 时，计算

$$G = \sum_{k=\tau}^{\min(\tau+n-1, T-1)} \gamma^{k-\tau} R_{k+1}.$$

4. 若 $\tau + n < T$ ，补上 $\gamma^n Q(S_{\tau+n}, A_{\tau+n})$ 。
5. 更新

$$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha[G - Q(S_\tau, A_\tau)].$$

伪代码：Q-learning

1. 初始化 $Q(s, a)$ 。
2. 用行为策略选动作 A ，常用 ϵ -greedy。
3. 执行动作，得到 R, S' 。
4. 执行异策略最优 backup:

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)].$$

5. 令 $S \leftarrow S'$ ；最终策略为 $\pi(s) = \arg \max_a Q(s, a)$ 。

伪代码：Dyna-Q

1. 初始化 $Q(s, a)$ 和模型 Model。
2. 真实交互：用 ϵ -greedy 选 A ，执行得到 R, S' 。
3. 用真实经验做 Q-learning 更新:

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)].$$

4. 学习模型：Model(S, A) $\leftarrow (R, S')$ 。
5. 规划 n 次：随机取旧 $(\tilde{S}, \tilde{A})$ ，由模型给出 $(\tilde{R}, \tilde{S}')$ ，再做同样的 Q-learning 更新。

17.4 DQN 系列

伪代码：DQN

1. 初始化在线网络 $Q(\cdot, \cdot; \theta)$ 、目标网络 $Q(\cdot, \cdot; \theta^-)$ 和 replay buffer $\mathcal{D}$ 。
2. 用 ϵ -greedy 与环境交互，存储 (s, a, r, s', d) 。
3. 从 $\mathcal{D}$ 采样 minibatch。
4. 计算目标

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-).$$

5. 最小化

$$(y - Q(s, a; \theta))^2.$$

6. 每隔 C 步更新目标网络 $\theta^- \leftarrow \theta$ 。

伪代码：Double DQN

1. 从 replay buffer 采样 (s, a, r, s', d) 。
2. 在线网络选动作：

$$a^* = \arg \max_{a'} Q(s', a'; \theta).$$

3. 目标网络评估动作：

$$y = r + \gamma(1 - d) Q(s', a^*; \theta^-).$$

4. 最小化 $(y - Q(s, a; \theta))^2$ 更新在线网络。

伪代码：Dueling DQN

1. 用共享网络提取状态特征。
2. 分成价值分支 $V(s)$ 和优势分支 $A(s, a)$ 。
3. 合并输出

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a').$$

4. 训练目标仍可使用 DQN 或 Double DQN 的 TD target。

17.5 策略梯度与 Actor-Critic

伪代码：REINFORCE

1. 初始化策略参数 θ 。
2. 用 π_θ 采样完整 episode。
3. 对每个时间步计算

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}.$$

4. 更新

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

5. 若有 baseline, 把 G_t 换成 $G_t - b(S_t)$ 。

伪代码：一步 Actor-Critic

1. 初始化 actor 参数 θ 和 critic 参数 w 。
2. 按 π_θ 选动作, 交互得到 R, S' 。
3. 计算

$$\delta = R + \gamma V_w(S') - V_w(S).$$

4. critic 更新:

$$w \leftarrow w + \beta \delta \nabla_w V_w(S).$$

5. actor 更新:

$$\theta \leftarrow \theta + \alpha \delta \nabla_\theta \log \pi_\theta(A | S).$$

17.6 TRPO 与 PPO

伪代码：TRPO

1. 用旧策略 $\pi_{\theta_{\text{old}}}$ 采样一批轨迹。
2. 估计优势 $\hat{A}_t$ 。
3. 最大化替代目标

$$\mathbb{E}_t \left[\frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)} \hat{A}_t \right]$$

同时满足平均 KL 约束。

4. 用共轭梯度近似求自然梯度方向 $H^{-1}g$ 。
5. 用线搜索选择满足 KL 约束且提升目标的步长。

伪代码：PPO-Clip

1. 用旧策略 $\pi_{\theta_{\text{old}}}$ 收集一批轨迹。
2. 计算回报目标和优势 $\hat{A}_t$ 。
3. 对 minibatch 计算比率

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

4. 最大化裁剪目标

$$\mathbb{E}_t [\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)].$$

5. 同时训练价值函数, 并可加入熵奖励; 对同一批数据优化 K 轮后重新采样。

18 Python 语言伪代码：考试默写版

本节保留 Python 语法风格, 但仍是伪代码。约定: `env.step` 表示执行动作并返回下一状态、奖励和终止标志; `epsilon_greedy` 表示按 ϵ -greedy 选动作; `model` 表示环境模型或学习到的模型。

18.1 动态规划：策略评估、策略迭代、价值迭代

```
1 def policy_evaluation(states, actions, P, pi, gamma, theta):
2     # P[s][a] = list of (prob, s_next, reward)
3     V = {s: 0.0 for s in states}
4     while True:
5         delta = 0.0
6         for s in states:
7             old_v = V[s]
8             new_v = 0.0
9             for a in actions[s]:
10                 for prob, s_next, r in P[s][a]:
11                     new_v += pi[s][a] * prob * (r + gamma * V[s_next])
12             V[s] = new_v
13             delta = max(delta, abs(old_v - V[s]))
14         if delta < theta:
15             break
16     return V
```

```
1 def policy_iteration(states, actions, P, gamma, theta):
2     pi = init_random_policy(states, actions)
3     V = {s: 0.0 for s in states}
4
5     while True:
6         V = policy_evaluation(states, actions, P, pi, gamma, theta)
7         stable = True
8
9         for s in states:
10             old_a = argmax_dict(pi[s])
11             q_values = {}
12             for a in actions[s]:
13                 q_values[a] = sum(
14                     prob * (r + gamma * V[s_next])
15                     for prob, s_next, r in P[s][a]
16                 )
17             best_a = argmax_dict(q_values)
18             pi[s] = one_hot_policy(actions[s], best_a)
19             if best_a != old_a:
20                 stable = False
21
22         if stable:
23             break
24     return pi, V
```

```
1 def value_iteration(states, actions, P, gamma, theta):
2     V = {s: 0.0 for s in states}
3
4     while True:
5         delta = 0.0
6         for s in states:
7             old_v = V[s]
8             q_values = []
9             for a in actions[s]:
10                 q = sum(
11                     prob * (r + gamma * V[s_next])
12                     for prob, s_next, r in P[s][a]
13                 )
14                 q_values.append(q)
15             V[s] = max(q_values)
16             delta = max(delta, abs(old_v - V[s]))
17         if delta < theta:
18             break
```

```
19
20 pi = {}
21 for s in states:
22     pi[s] = argmax_action(actions[s], lambda a: sum(
23         prob * (r + gamma * V[s_next])
24         for prob, s_next, r in P[s][a]
25     ))
26 return pi, V
```

18.2 MC：蒙特卡洛预测与控制

```
1 def first_visit_mc_prediction(env, pi, gamma, alpha, num_episodes):
2     V = defaultdict(float)
3
4     for episode in range(num_episodes):
5         trajectory = []
6         s = env.reset()
7         done = False
8         while not done:
9             a = sample_from_policy(pi, s)
10            s_next, r, done = env.step(a)
11            trajectory.append((s, a, r))
12            s = s_next
13
14        G = 0.0
15        visited_states = set()
16        for t in reversed(range(len(trajectory))):
17            s_t, a_t, r_tp1 = trajectory[t]
18            G = gamma * G + r_tp1
19            if s_t not in visited_states:
20                visited_states.add(s_t)
21                V[s_t] += alpha * (G - V[s_t])
22    return V
```

```
1 def on_policy_mc_control(env, gamma, alpha, eps, num_episodes):
2     Q = defaultdict(float)
3     pi = make_epsilon_greedy_policy(Q, eps)
4
5     for episode in range(num_episodes):
6         trajectory = []
7         s = env.reset()
8         done = False
9         while not done:
10            a = epsilon_greedy(Q, s, eps)
11            s_next, r, done = env.step(a)
12            trajectory.append((s, a, r))
13            s = s_next
14
15        G = 0.0
16        visited_pairs = set()
17        for t in reversed(range(len(trajectory))):
18            s_t, a_t, r_tp1 = trajectory[t]
19            G = gamma * G + r_tp1
20            if (s_t, a_t) not in visited_pairs:
21                visited_pairs.add((s_t, a_t))
22                Q[s_t, a_t] += alpha * (G - Q[s_t, a_t])
23
24        pi = make_epsilon_greedy_policy(Q, eps)
25    return Q, pi
```

18.3 TD(0)、SARSA、n-step SARSA、Q-learning

```
1 def td0_prediction(env, pi, gamma, alpha, num_episodes):
2     V = defaultdict(float)
3
4     for episode in range(num_episodes):
5         s = env.reset()
6         done = False
7         while not done:
8             a = sample_from_policy(pi, s)
9             s_next, r, done = env.step(a)
10            target = r if done else r + gamma * V[s_next]
11            V[s] += alpha * (target - V[s])
12            s = s_next
13     return V
```

```
1 def sarsa(env, gamma, alpha, eps, num_episodes):
2     Q = defaultdict(float)
3
4     for episode in range(num_episodes):
5         s = env.reset()
6         a = epsilon_greedy(Q, s, eps)
7         done = False
8         while not done:
9             s_next, r, done = env.step(a)
10            if done:
11                target = r
12            else:
13                a_next = epsilon_greedy(Q, s_next, eps)
14                target = r + gamma * Q[s_next, a_next]
15            Q[s, a] += alpha * (target - Q[s, a])
16            if not done:
17                s, a = s_next, a_next
18     return Q
```

```
1 def n_step_sarsa(env, gamma, alpha, eps, n, num_episodes):
2     Q = defaultdict(float)
3
4     for episode in range(num_episodes):
5         S, A, R = [], [], [0.0]
6         S.append(env.reset())
7         A.append(epsilon_greedy(Q, S[0], eps))
8         T = float("inf")
9         t = 0
10
11        while True:
12            if t < T:
13                s_next, r, done = env.step(A[t])
14                R.append(r)
15                S.append(s_next)
16                if done:
17                    T = t + 1
18                else:
19                    A.append(epsilon_greedy(Q, s_next, eps))
20
21            tau = t - n + 1
22            if tau >= 0:
23                G = 0.0
24                upper = min(tau + n, T)
25                for i in range(tau + 1, int(upper) + 1):
26                    G += (gamma ** (i - tau - 1)) * R[i]
27                if tau + n < T:
```

```

28         G += (gamma ** n) * Q[S[tau + n], A[tau + n]]
29         Q[S[tau], A[tau]] += alpha * (G - Q[S[tau], A[tau]])
30
31         if tau == T - 1:
32             break
33         t += 1
34     return Q

```

```

1 def q_learning(env, gamma, alpha, eps, num_episodes):
2     Q = defaultdict(float)
3
4     for episode in range(num_episodes):
5         s = env.reset()
6         done = False
7         while not done:
8             a = epsilon_greedy(Q, s, eps)      # behavior policy
9             s_next, r, done = env.step(a)
10            if done:
11                target = r
12            else:
13                target = r + gamma * max(Q[s_next, a2] for a2 in env.actions(s_next))
14            Q[s, a] += alpha * (target - Q[s, a])
15            s = s_next
16    return Q

```

18.4 Dyna-Q

```

1 def dyna_q(env, gamma, alpha, eps, planning_steps, num_steps):
2     Q = defaultdict(float)
3     model = {}          # model[(s, a)] = (r, s_next, done)
4     seen_pairs = []
5     s = env.reset()
6
7     for step in range(num_steps):
8         a = epsilon_greedy(Q, s, eps)
9         s_next, r, done = env.step(a)
10
11        # direct RL update
12        best_next = 0.0 if done else max(Q[s_next, a2] for a2 in env.actions(s_next))
13        Q[s, a] += alpha * (r + gamma * best_next - Q[s, a])
14
15        # model learning
16        if (s, a) not in model:
17            seen_pairs.append((s, a))
18        model[(s, a)] = (r, s_next, done)
19
20        # planning updates from simulated experience
21        for k in range(planning_steps):
22            s_sim, a_sim = random_choice(seen_pairs)
23            r_sim, s_sim_next, done_sim = model[(s_sim, a_sim)]
24            best_sim = 0.0 if done_sim else max(
25                Q[s_sim_next, a2] for a2 in env.actions(s_sim_next)
26            )
27            Q[s_sim, a_sim] += alpha * (
28                r_sim + gamma * best_sim - Q[s_sim, a_sim]
29            )
30
31        s = env.reset() if done else s_next
32    return Q, model

```

18.5 DQN、Double DQN、Dueling DQN

```

1 def train_dqn(env, Q_net, target_net, optimizer, replay, gamma, eps, target_period):
2     s = env.reset()
3     for step in range(total_steps):
4         a = epsilon_greedy_network(Q_net, s, eps)
5         s_next, r, done = env.step(a)
6         replay.add(s, a, r, s_next, done)
7         s = env.reset() if done else s_next
8
9         batch = replay.sample(batch_size)
10        q_sa = Q_net(batch.s).gather(actions=batch.a)
11        with no_grad():
12            q_next = target_net(batch.s_next).max(dim=1)
13            y = batch.r + gamma * (1 - batch.done) * q_next
14            loss = mean_square_error(q_sa, y)
15
16        optimizer.zero_grad()
17        loss.backward()
18        optimizer.step()
19
20        if step % target_period == 0:
21            target_net.load_state_dict(Q_net.state_dict())

```

```

1 def train_double_dqn(env, Q_net, target_net, optimizer, replay, gamma, eps):
2     batch = replay.sample(batch_size)
3     q_sa = Q_net(batch.s).gather(actions=batch.a)
4
5     with no_grad():
6         # online network selects action; target network evaluates it
7         a_star = Q_net(batch.s_next).argmax(dim=1)
8         q_next = target_net(batch.s_next).gather(actions=a_star)
9         y = batch.r + gamma * (1 - batch.done) * q_next
10
11    loss = mean_square_error(q_sa, y)
12    optimizer.zero_grad()
13    loss.backward()
14    optimizer.step()

```

```

1 class DuelingQNetwork:
2     def forward(self, s):
3         z = shared_feature_layers(s)
4         V = value_head(z) # shape: [batch, 1]
5         A = advantage_head(z) # shape: [batch, num_actions]
6         Q = V + A - mean(A, dim=1, keepdim=True)
7         return Q
8
9 # Training target can be the same as DQN or Double DQN.

```

18.6 REINFORCE 与 Actor-Critic

```

1 def reinforce(env, policy_net, optimizer, gamma, num_episodes):
2     for episode in range(num_episodes):
3         log_probs, rewards = [], []
4         s = env.reset()
5         done = False
6
7         while not done:
8             dist = policy_net.action_distribution(s)
9             a = dist.sample()

```

```

10     log_probs.append(dist.log_prob(a))
11     s, r, done = env.step(a)
12     rewards.append(r)
13
14     G = 0.0
15     returns = []
16     for r in reversed(rewards):
17         G = r + gamma * G
18         returns.insert(0, G)
19
20     loss = 0.0
21     for log_p, G_t in zip(log_probs, returns):
22         loss += -log_p * G_t           # gradient ascent on J
23
24     optimizer.zero_grad()
25     loss.backward()
26     optimizer.step()

```

```

1 def actor_critic(env, actor, critic, actor_opt, critic_opt, gamma, num_steps):
2     s = env.reset()
3     for step in range(num_steps):
4         dist = actor.action_distribution(s)
5         a = dist.sample()
6         s_next, r, done = env.step(a)
7
8         V_s = critic(s)
9         V_next = 0.0 if done else critic(s_next).detach()
10        delta = r + gamma * V_next - V_s
11
12        critic_loss = delta.pow(2)
13        actor_loss = -dist.log_prob(a) * delta.detach()
14
15        critic_opt.zero_grad()
16        critic_loss.backward()
17        critic_opt.step()
18
19        actor_opt.zero_grad()
20        actor_loss.backward()
21        actor_opt.step()
22
23        s = env.reset() if done else s_next

```

18.7 TRPO 与 PPO

```

1 def trpo(policy, value_fn, trajectories, max_kl, damping):
2     # 1. estimate advantage
3     advantages = compute_gae(trajectories, value_fn)
4     old_logp = policy.log_prob(trajectories.states, trajectories.actions).detach()
5
6     # 2. surrogate objective
7     def surrogate_loss(theta):
8         set_policy_params(policy, theta)
9         logp = policy.log_prob(trajectories.states, trajectories.actions)
10        ratio = exp(logp - old_logp)
11        return -mean(ratio * advantages)      # minimize negative objective
12
13    # 3. natural gradient direction
14    g = gradient_of_surrogate_at_old_policy()
15    Hx = lambda x: fisher_vector_product(policy, trajectories.states, x, damping)
16    step_dir = conjugate_gradient(Hx, g)
17

```

```

18 # 4. scale step to satisfy KL trust region
19 scale = sqrt(2 * max_kl / dot(step_dir, Hx(step_dir)))
20 full_step = scale * step_dir
21
22 # 5. line search
23 old_theta = get_policy_params(policy)
24 for frac in [1.0, 0.5, 0.25, 0.125]:
25     new_theta = old_theta + frac * full_step
26     set_policy_params(policy, new_theta)
27     if mean_kl(policy, trajectories.states) <= max_kl and improves_objective():
28         break
29 else:
30     set_policy_params(policy, old_theta)

```

```

1 def ppo_clip(env, actor_critic, optimizer, gamma, lam, clip_eps, train_epochs):
2     for iteration in range(num_iterations):
3         trajectories = collect_rollouts(env, actor_critic.policy)
4         returns, advantages = compute_gae_returns(trajectories, gamma, lam)
5         old_logp = trajectories.logp.detach()
6
7         for epoch in range(train_epochs):
8             for batch in minibatches(trajectories, returns, advantages):
9                 logp, entropy, value = actor_critic.evaluate(batch.s, batch.a)
10                ratio = exp(logp - batch.old_logp)
11
12                unclipped = ratio * batch.advantage
13                clipped = clip(ratio, 1 - clip_eps, 1 + clip_eps) * batch.advantage
14                policy_loss = -mean(minimum(unclipped, clipped))
15                value_loss = mean_square_error(value, batch.return_)
16                entropy_bonus = mean(entropy)
17
18                loss = policy_loss + c1 * value_loss - c2 * entropy_bonus
19                optimizer.zero_grad()
20                loss.backward()
21                optimizer.step()

```

19 算法之间的递进关系

19.1 从 Bellman 到价值学习

DP 是 Bellman 方程的直接求解器。已知模型时，Bellman 期望方程给出策略评估，Bellman 最优化方程给出价值迭代。未知模型时，无法计算

$$\sum_{s',r} p(s',r | s,a)[r + \gamma V(s')],$$

于是改用样本替代期望：

$$R + \gamma V(S')$$

这就是 TD。若不用 bootstrap，而是等完整回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots,$$

就是 MC。

19.2 从 TD 到控制算法

TD 预测只评估固定策略。要控制，需要学习动作价值 $Q(s, a)$:

$$Q \leftarrow Q + \alpha(\text{target} - Q).$$

若 target 使用实际下一动作:

$$\text{target} = R + \gamma Q(S', A'),$$

得到 SARSA, 同策略。若 target 使用最优下一动作:

$$\text{target} = R + \gamma \max_{a'} Q(S', a'),$$

得到 Q-learning, 异策略。把 Q 表换成神经网络, 就是 DQN。

19.3 从 Q-learning 到 DQN 系列

表格 Q-learning 的更新:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right].$$

DQN 将 $Q(S, A)$ 替换成 $Q(S, A; \theta)$, 用均方误差训练。目标网络稳定目标:

$$y = R + \gamma \max_{a'} Q(S', a'; \theta^-).$$

Double DQN 发现 max 会放大估计噪声, 所以拆成:

$$\arg \max_{a'} Q(S', a'; \theta) \quad \text{和} \quad Q(S', a^*; \theta^-).$$

Dueling DQN 则从表达能力角度改进, 把

$$Q(s, a)$$

拆成

$$V(s) + A(s, a)$$

使网络更容易学到“这个状态本身好不好”和“某动作相对好不好”。

19.4 从价值法到策略梯度

价值法依赖 $\arg \max_a Q(s, a)$, 在连续动作空间中求 $\arg \max$ 很难; 且有时需要显式随机策略。于是直接优化策略:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta).$$

REINFORCE 用完整回报估计梯度:

$$\nabla J \approx \sum_t G_t \nabla \log \pi_{\theta}(A_t | S_t).$$

为降低方差, 引入 baseline/critic:

$$G_t - V(S_t) \quad \text{或} \quad \delta_t$$

得到 Actor-Critic。普通策略梯度步长太大可能毁掉策略, 于是 TRPO 用 KL 约束:

$$\bar{D}_{\text{KL}}(\pi_{\text{old}}, \pi_{\theta}) \leq \delta.$$

PPO 用裁剪比率近似该约束:

$$\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t).$$

19.5 Dyna-Q 的位置

若说 Q-learning 完全不使用模型，DP 完全依赖真实模型，Dyna-Q 位于中间：

$$\text{真实经验} \Rightarrow \begin{cases} \text{更新 } Q \\ \text{更新模型 } \hat{M} \\ \text{用 } \hat{M} \text{ 生成模拟经验再更新 } Q \end{cases}$$

所以它体现了 Sutton 经典框架：

direct RL + model learning + planning.

20 常考概念辨析

20.1 同策略与异策略

行为策略 b ：用来采样数据的策略。目标策略 π ：被评估或优化的策略。

$$b = \pi \Rightarrow \text{同策略}, \quad b \neq \pi \Rightarrow \text{异策略}.$$

SARSA 是同策略，因为更新目标中的 A' 是行为策略实际采样的动作：

$$R + \gamma Q(S', A').$$

Q-learning 是异策略，因为更新目标中的动作来自贪心选择：

$$R + \gamma \max_{a'} Q(S', a'),$$

即使行为策略为了探索选择了非贪心动作。

20.2 在线与离线

在线学习：数据一产生就可更新，例如 TD、SARSA、Q-learning。

离线学习有两层含义：

- episode 内离线：如 MC 等完整 episode 结束后再更新；
- batch/offline RL：完全使用固定历史数据集，不再与环境交互。

考试中若问 MC 是否在线，一般答：原始 MC 需要 episode 完整结束后更新，不是逐步在线 TD；但增量 MC 可在每个 episode 后在线地更新统计量。

20.3 有模型与无模型

有模型算法使用或学习

$$p(s', r \mid s, a)$$

进行规划。DP 使用已知真实模型；Dyna-Q 学习近似模型。无模型算法不显式估计转移概率，只用样本更新价值或策略。

20.4 基于价值与基于策略

基于价值：

$$\text{先学 } V/Q, \quad \pi(s) = \arg \max_a Q(s, a).$$

代表：DP、MC control、TD control、SARSA、Q-learning、DQN。

基于策略：

$$\text{直接学 } \pi_\theta(a | s).$$

代表：REINFORCE、TRPO、PPO。Actor-Critic 同时有 actor 和 critic，是策略梯度家族中的核心结构。

20.5 探索与利用

ϵ -greedy：

$$a = \begin{cases} \arg \max_a Q(s, a), & \text{概率 } 1 - \epsilon, \\ \text{随机动作}, & \text{概率 } \epsilon. \end{cases}$$

softmax 策略：

$$\pi(a | s) = \frac{\exp(Q(s, a)/\tau)}{\sum_b \exp(Q(s, b)/\tau)}.$$

τ 越大越随机， τ 越小越接近贪心。

21 答题模板与易错点

21.1 答算法题的通用模板

若考试要求描述某算法，按以下顺序写：

1. 问题设定：有无模型、同/异策略、价值/策略、在线/离线。
2. 核心目标：Bellman 期望、Bellman 最优、策略梯度或 surrogate objective。
3. 更新公式：写清 target、误差、参数更新。
4. 算法流程：初始化、采样、计算目标、更新、改进策略或更新网络。
5. 优缺点：偏差方差、稳定性、样本效率、适用空间。

21.2 常见易错点

- SARSA 的 target 是 $Q(S', A')$ ，不是 $\max_{a'} Q(S', a')$ 。
- Q-learning 是 off-policy，即使行为策略也是 ϵ -greedy。
- MC 不 bootstrap，TD bootstrap。
- 策略迭代有显式策略评估和策略改进；价值迭代直接做最优 backup。
- Dyna-Q 不是纯 DP，因为模型来自经验；也不是纯 model-free，因为它用模型做模拟规划。
- DQN 的 target 用目标网络 θ^- ，训练的是在线网络 θ 。
- Double DQN 的关键是动作选择和动作评估分离。

- Dueling DQN 的关键是网络结构分解，不是新的 Bellman target。
- REINFORCE 用完整回报，方差高；Actor-Critic 用 critic 估计优势，方差低但有偏。
- TRPO 是显式 KL 约束；PPO 是 clipped surrogate 的近似。

22 公式速查表

MDP 与价值

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

$$v_{\pi}(s) = \sum_a \pi(a \mid s) q_{\pi}(s, a)$$

Bellman 期望

$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]$$

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \right]$$

Bellman 最优

$$v_*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]$$

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma \max_{a'} q_*(s', a')]$$

DP

$$V_{k+1} = T_{\pi} V_k$$

$$V_{k+1} = T_* V_k$$

$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')]$$

MC

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)]$$

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

TD

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n})$$

SARSA

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$$

$$Q \leftarrow Q + \alpha \delta_t$$

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q(S_{t+n}, A_{t+n})$$

Q-learning

$$\delta_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)$$

$$Q \leftarrow Q + \alpha \delta_t$$

DQN

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-)$$

$$L(\theta) = \mathbb{E}[(y - Q(s, a; \theta))^2]$$

$$y_{\text{Double}} = r + \gamma(1 - d)Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$$

$$Q = V + \left(A - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$$

Actor-Critic

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)$$

$$w \leftarrow w + \beta \delta_t \nabla_w V_w(S_t)$$

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

Policy Gradient

$$\nabla J(\theta) = \mathbb{E} \left[\sum_t \nabla \log \pi_{\theta}(A_t | S_t) G_t \right]$$

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

$$\theta \leftarrow \theta + \alpha \hat{A}_t \nabla \log \pi_{\theta}(A_t | S_t)$$

TRPO/PPO

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}$$

$$\max_{\theta} \mathbb{E}[r_t(\theta) \hat{A}_t], \quad \mathbb{E}[D_{\text{KL}}(\pi_{\text{old}} \| \pi_{\theta})] \leq \delta$$

$$L^{\text{CLIP}} = \mathbb{E} \left[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

23 最后串讲：考场叙述版

强化学习从 MDP 开始。MDP 用状态、动作、转移概率、奖励和折扣因子描述序贯决策问题。策略 π 决定在状态下如何选择动作，目标是最大化折扣回报 G_t 。为了评价策略，引入状态价值 $v_{\pi}(s)$ 和动作价值 $q_{\pi}(s, a)$ 。由回报递推 $G_t = R_{t+1} + \gamma G_{t+1}$ 可得 Bellman 期望方程；对动作取最大可得 Bellman 最优化方程。

如果模型已知，就能直接计算 Bellman 方程中的期望，这就是动态规划。策略迭代先评估当前策略，再做贪心改进；价值迭代直接反复做最优 Bellman backup。二者都依赖完整模型和状态动作遍历。

如果模型未知，就只能从经验学习。MC 用完整回报 G_t 更新价值，不 bootstrap，方差大但直观；TD 用 $R + \gamma V(S')$ 更新价值，bootstrap，能一步一更新。控制问题中要比较动作，因此学习 $Q(s, a)$ 。SARSA 使用实际下一动作 A' ，是同策略；Q-learning 使用 $\max_{a'} Q(S', a')$ ，是异策略，直接逼近最优动作价值。

当 Q 表太大时，用神经网络近似 $Q(s, a; \theta)$ 得到 DQN。DQN 继承 Q-learning 的目标，但加入经验回放和目标网络稳定训练。Double DQN 将动作选择和动作评估分离，缓解过估计；Dueling DQN 将网络输出分解为状态价值和优势函数，提升表达效率。

价值法适合离散动作，但连续动作下 $\arg \max_a Q(s, a)$ 难求，而且有时需要显式随机策略。因此直接参数化 $\pi_{\theta}(a | s)$ ，通过策略梯度优化期望回报。REINFORCE 用完整回报乘以 $\nabla \log \pi$ 做更新，简单但方差大。Actor-Critic 引入 critic 估计价值或优势，用 TD 误差指导 actor，降低方差并允许在线更新。TRPO 进一步限制新旧策略 KL 距离，避免策略一步变太多；PPO 用裁剪重要性比率近似这个限制，实现更简单，是常用深度策略优化算法。

Dyna-Q 则把价值学习和模型规划连接起来：真实交互既更新 Q ，也更新模型；随后用模型生成模拟经验做额外 Q-learning 更新。它说明 model-free 和 model-based 不是完全割裂的两端，而可以组合起来提高样本效率。

考场上可把所有算法统一写成一句话：RL 的核心是在 MDP 中求最大长期回报；Bellman 方程给出价值递推；DP 在已知模型时精确做期望 backup；MC/TD 在未知模型时用样本近似 backup；SARSA/Q-

learning/DQN 用动作价值做控制；REINFORCE/A-C/TRPO/PPO 直接优化策略；Dyna-Q 用学到的模型把真实经验扩展成规划更新。

24 高频简答题预测与参考答案

本节按“简答题”风格整理。考试作答时建议每题按“概念定义 → 核心公式 → 算法流程 → 对比/递进 → 优缺点或应用场景”的顺序写。

24.1 题 1：什么是 MDP？写出 Bellman 期望方程和 Bellman 最优化方程，并说明它们在强化学习算法中的作用。

参考答案：

马尔可夫决策过程（MDP）可记为

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma),$$

其中 $\mathcal{S}$ 为状态空间， $\mathcal{A}$ 为动作空间， $p(s', r | s, a)$ 为转移和奖励的联合概率， γ 为折扣因子。马尔可夫性表示未来只依赖当前状态和动作，与过去历史无关。

固定策略 π 下，状态价值函数满足 Bellman 期望方程：

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')].$$

动作价值函数满足：

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right].$$

最优价值函数满足 Bellman 最优化方程：

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')],$$

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')].$$

Bellman 期望方程用于策略评估，回答“当前策略好不好”；Bellman 最优化方程用于最优控制，回答“最优策略应该怎么选”。DP 直接计算 Bellman 期望；TD、SARSA、Q-learning 用样本近似 Bellman backup；DQN 用神经网络近似 q_* ；策略梯度算法虽不直接求 Bellman 最优方程，但 critic 仍常用 Bellman 误差训练价值函数。

24.2 题 2：比较策略迭代和价值迭代。二者如何从 Bellman 方程递进而来？

参考答案：

策略迭代和价值迭代都属于动态规划，前提是已知环境模型 $p(s', r | s, a)$ 。策略迭代由两个步骤交替组成：策略评估和策略改进。策略评估使用 Bellman 期望方程：

$$V(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')],$$

求当前策略的价值；策略改进再做一步贪心：

$$\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

价值迭代把策略评估压缩为一步，直接使用 Bellman 最优 backup：

$$V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

二者区别是：策略迭代有明确的“评估当前策略 → 改进策略”过程，每轮计算量较大但轮数通常较少；价值迭代直接逼近 V_* ，每轮较便宜但可能需要更多轮。二者共同点是都需要模型、都要遍历状态动作空间、都基于 Bellman 方程的不动点思想。

24.3 题 3：比较 MC 和 TD。为什么说 TD 连接了 MC 与 DP？

参考答案：

MC 和 TD 都是无模型预测方法。MC 使用完整 episode 的真实回报

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

作为目标：

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)].$$

TD 使用一步 bootstrap 目标：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)].$$

区别在于：MC 不 bootstrap，通常无偏但方差大，必须等 episode 结束才能得到完整回报；TD bootstrap，目标中含当前估计值，因此有偏但方差较小，可以每一步在线更新，也适合 continuing 任务。

TD 连接 MC 与 DP 的原因是：它像 MC 一样从真实样本中学习，不需要模型；又像 DP 一样使用 Bellman 递推，用 $R + \gamma V(S')$ 作为估计目标。进一步地， n -step TD 的目标

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n})$$

当 $n = 1$ 时是 TD(0)，当 n 到 episode 末尾时接近 MC，因此 n -step 方法明确体现了二者之间的连续过渡。

24.4 题 4：比较 SARSA 和 Q-learning。为什么一个是同策略，一个是异策略？

参考答案：

SARSA 和 Q-learning 都是无模型 TD 控制算法，都学习动作价值函数 $Q(s, a)$ ，但更新目标不同。SARSA 的目标是：

$$R + \gamma Q(S', A'),$$

更新为

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)].$$

其中 A' 是按照当前行为策略实际选出的下一动作，所以 SARSA 学的是当前实际执行策略的价值，是同策略算法。

Q-learning 的目标是：

$$R + \gamma \max_{a'} Q(S', a'),$$

更新为

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)].$$

它采样时可以用 ϵ -greedy 探索，但更新目标假设下一步使用贪心动作，因此行为策略和目标策略不同，是异策略算法。

从风险偏好看，SARSA 会把探索动作的风险计入价值估计，所以更保守；Q-learning 学习贪心最优策略，更激进。二者的递进关系是：TD 预测学习 V^π ，控制任务需要学习 Q ；若目标用实际下一动作得到 SARSA，若目标用最优下一动作得到 Q-learning。

24.5 题 5：从 Q-learning 到 DQN、Double DQN、Dueling DQN 的改进脉络是什么？

参考答案：

Q-learning 是表格方法，更新为

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_{a'} Q(S', a') - Q(S, A)].$$

当状态空间很大或输入为图像时，表格无法存储，于是 DQN 用神经网络 $Q(s, a; \theta)$ 近似 Q 函数。DQN 的目标为：

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-),$$

损失函数为

$$L(\theta) = \mathbb{E}[(y - Q(s, a; \theta))^2].$$

DQN 的关键稳定技巧是经验回放和目标网络。经验回放打破样本相关性并复用数据；目标网络用较旧参数 θ^- 计算目标，减轻训练目标不断移动的问题。

Double DQN 改进的是过估计问题。普通 DQN 中 $\max$ 同时选择和评估动作，容易把噪声也最大化。Double DQN 将二者分离：

$$\begin{aligned} a^* &= \arg \max_{a'} Q(s', a'; \theta), \\ y &= r + \gamma(1 - d) Q(s', a^*; \theta^-). \end{aligned}$$

Dueling DQN 改进的是网络结构，把 Q 分解为状态价值和优势函数：

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a').$$

这样网络能分别学习“状态本身好不好”和“动作相对好不好”。因此改进脉络是：DQN 解决大状态空间问题；Double DQN 减少过估计；Dueling DQN 提高价值表达效率。

24.6 题 6：比较 REINFORCE 和 Actor-Critic。为什么要从纯策略梯度发展到 Actor-Critic？

参考答案：

REINFORCE 是最基本的策略梯度算法，直接优化参数化策略 $\pi_\theta(a | s)$ 。策略梯度为：

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(A_t | S_t) G_t \right],$$

更新为

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

REINFORCE 的优点是简单、无模型、直接优化策略，适合随机策略和连续动作；缺点是使用完整回报 G_t ，方差大，通常要等 episode 结束后更新，样本效率低。

Actor-Critic 引入 critic 估计价值函数，用 TD 误差或优势函数指导 actor。一步 A-C 中：

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t),$$

critic 更新：

$$w \leftarrow w + \beta \delta_t \nabla_w V_w(S_t),$$

actor 更新：

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

其中 δ_t 近似优势函数 $A^{\pi}(s, a)$ 。从 REINFORCE 到 A-C 的递进是：用 critic 作为 baseline 或优势估计，降低策略梯度方差，并允许在线或小批量更新。但 critic 近似也会引入偏差，因此 A-C 是以一定偏差换取更低方差和更好样本效率。

24.7 题 7：比较 TRPO 和 PPO。PPO 是如何继承并简化 TRPO 思想的？

参考答案：

TRPO 和 PPO 都属于同策略、无模型、基于策略的深度强化学习算法，核心目标是避免策略一次更新过大导致性能崩溃。TRPO 使用显式 KL 约束：

$$\begin{aligned} & \max_{\theta} \mathbb{E}_t[r_t(\theta) \hat{A}_t], \\ \text{s.t. } & \mathbb{E}_t[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | S_t) \| \pi_{\theta}(\cdot | S_t))] \leq \delta, \end{aligned}$$

其中

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

TRPO 通过自然梯度、共轭梯度和线搜索近似求解，理论稳定性强，但实现复杂、计算成本高。

PPO 继承了“限制新旧策略差异”的思想，但不用复杂约束优化，而是使用裁剪目标：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)].$$

当优势为正时，PPO 不允许动作概率被过度增大；当优势为负时，不允许动作概率被过度降低。这样用简单的 SGD/Adam 就能训练，并且可对同一批数据做多轮 minibatch 更新。简言之：TRPO 是显式 KL 约束的信赖域优化，稳定但复杂；PPO 是用比率裁剪近似信赖域，稳定性较好且工程实现简单。

24.8 题 8：Dyna-Q 为什么说是 model-free 和 model-based 的结合？与 DP 和 Q-learning 分别有什么关系？

参考答案：

Dyna-Q 同时包含真实经验学习、模型学习和规划。每次真实交互得到 (S, A, R, S') 后，它先做一次 Q-learning 更新：

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_{a'} Q(S', a') - Q(S, A)].$$

然后把经验存入模型：

$$\text{Model}(S, A) \leftarrow (R, S').$$

接着从已见过的 (s, a) 中抽样，用模型生成模拟经验，再做若干次 Q-learning 更新。

它与 Q-learning 的关系是：真实更新和模拟更新的核心公式都是 Q-learning 的 off-policy TD control。它与 DP 的关系是：Dyna-Q 使用模型进行 planning，但这个模型不是预先已知的真实转移模型，而是从经验中学习得到的近似模型。它的优点是提高样本效率，因为一次真实交互可以带来多次模拟更新；缺点是模型错误可能传播到价值函数，非平稳环境中旧模型还可能误导学习。

24.9 题 9：请从“有模型/无模型、同策略/异策略、在线/离线、基于价值/基于策略”的角度对主要算法分类。

参考答案：

有模型算法使用或学习环境模型 $p(s', r | s, a)$ 。DP 是典型有模型算法，因为它需要已知转移概率和奖励；Dyna-Q 是学习模型再规划，处在 model-based 和 model-free 的中间。无模型算法不显式估计转移概率，包括 MC、TD、SARSA、Q-learning、DQN、REINFORCE、A-C、TRPO、PPO。

同策略算法中，行为策略和目标策略相同。SARSA 使用实际下一动作 A' ，所以是同策略；REINFORCE、A-C、TRPO、PPO 通常也按当前策略采样并优化当前策略，是同策略。异策略算法中，行为策略和目标策略不同。Q-learning 和 DQN 用探索策略采样，但目标为贪心策略，因此是异策略。

在线算法可以在交互过程中逐步更新，如 TD、SARSA、Q-learning、Dyna-Q、A-C。MC 通常需要完整 episode 后计算 G_t ，更偏 episode-end 更新。PPO/TRPO 通常先收集一批 trajectory，再进行 batch 更新。

基于价值的方法先学习 V 或 Q ，再用贪心策略选动作，如 DP、MC control、TD control、SARSA、Q-learning、DQN。基于策略的方法直接学习 $\pi_\theta(a | s)$ ，如 REINFORCE、TRPO、PPO。Actor-Critic 同时学习策略和价值函数，是策略梯度家族中结合价值估计的结构。

24.10 题 10：请串讲强化学习算法从 DP 到 PPO 的发展脉络。

参考答案：

强化学习的起点是 MDP 和 Bellman 方程。若模型已知，可以直接计算 Bellman 期望和 Bellman 最优方程，得到动态规划方法，包括策略迭代和价值迭代。它们理论清晰，但要求完整模型和状态动作遍历。

现实中模型通常未知，于是转向从经验中学习。MC 用完整回报估计价值，不 bootstrap；TD 用一步奖励加下一状态价值估计作为目标，能在线更新。控制问题中需要比较动作，因此从 V 转向 Q 。SARSA 使用实际下一动作，是同策略 TD 控制；Q-learning 使用最大动作价值，是异策略最优控制。

当状态空间变大时，表格 Q-learning 不再适用，DQN 用深度网络近似 $Q(s, a)$ ，并用经验回放和目标网络稳定训练。Double DQN 解决过估计，Dueling DQN 改进网络结构。

价值法在连续动作或需要随机策略时不够自然，于是发展出策略梯度。REINFORCE 直接用完整回报优化策略，但方差大；Actor-Critic 引入 critic 估计价值或优势，降低方差。为了避免策略更新过大，TRPO 使用 KL 信赖域约束；PPO 用裁剪目标近似信赖域，成为更易实现、更常用的深度策略优化方法。整个脉络可以概括为：从已知模型规划，到无模型价值学习，再到深度价值学习，最后到直接策略优化和稳定策略更新。

24.11 题 11：课程感悟题：学习强化学习后，你对人工智能或未来智能体研究有什么理解和启发？

参考答案：

强化学习给我的最大启发是：智能不仅是对已有数据进行拟合，更重要的是在环境交互中通过试错、反馈和长期目标进行决策。监督学习通常关注单步预测是否正确，而强化学习关注序贯决策中当前动作对未来结果的影响，因此更接近真实智能体面对复杂世界时的学习方式。

从课程内容看，MDP 和 Bellman 方程提供了一个非常清晰的数学框架：把复杂决策问题分解为状态、动作、奖励、转移和价值递推。DP、MC、TD、Q-learning、DQN、策略梯度等算法虽然形式不同，但本质上都在解决“如何估计长期价值”和“如何改进策略”这两个核心问题。这让我认识到，算法发展往往不是孤立的，而是围绕同一个核心矛盾不断改进：模型已知与未知、偏差与方差、探索与利用、稳定性与样本效率、价值估计与策略优化。

对未来人工智能研究而言，强化学习的重要意义在于它使智能体具备主动决策和长期规划能力。无论是机器人控制、自动驾驶、游戏智能体，还是大模型工具调用、多智能体协作和具身智能，都需要智能体根据环境反馈持续调整行为。但强化学习也面临挑战，例如样本效率低、训练不稳定、奖励设计困难、安全约束不足、现实环境迁移困难等。未来研究可能会更多结合模型学习、离线数据、世界模型、人类反馈和安全约束，使智能体既能高效学习，又能可靠地在真实环境中行动。

24.12 题 12：课程感悟题：请结合本课程算法，谈谈你对“探索与利用”和“稳定训练”的理解。

参考答案：

探索与利用是强化学习中的基本矛盾。利用是选择当前看来最优的动作以获得较高回报，探索是尝试不确定动作以发现更优策略。如果只利用，智能体可能陷入局部最优；如果过度探索，短期性能会下降。 ϵ -greedy、softmax 策略、策略熵奖励等方法都体现了探索与利用的平衡。SARSA 因为学习实际探索策略，会把探索风险计入价值；Q-learning 虽然行为上探索，但目标上学习贪心策略，体现了异策略学习的思想。

稳定训练也是深度强化学习的核心问题。传统表格方法中，Bellman backup 较清晰；但引入神经网络后，数据相关性、目标非平稳性和最大化偏差会导致训练不稳定。DQN 用经验回放和目标网络缓解这些问题；Double DQN 减少过估计；Dueling DQN 改善价值表达。策略梯度中，更新过大可能导致策略性能突然崩溃，因此 TRPO 用 KL 约束限制策略变化，PPO 用裁剪目标实现更简单的稳定更新。

因此，我的理解是：强化学习算法的改进往往不是单纯追求更复杂的模型，而是在探索效率、估计偏差、方差、训练稳定性和计算成本之间寻找平衡。这种思想对未来 AI 研究很重要，因为真实智能体不仅要“学得快”，还要“学得稳、行为安全、能长期优化”。

24.13 题 13：什么是策略改进定理？它为什么能保证策略迭代逐步变好？

参考答案：

策略改进定理说明：若对所有状态 s ，新策略 π' 满足

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s),$$

则

$$v_{\pi'}(s) \geq v_{\pi}(s), \quad \forall s.$$

也就是说，如果在每个状态下，新策略选择的动作按旧策略价值来看都不比旧策略差，那么从长期回报看，新策略也不会比旧策略差。

在策略迭代中，先通过策略评估得到 v_π ，再按一步贪心进行策略改进：

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')].$$

这个动作的价值必然不小于旧策略按概率平均得到的价值，因此满足策略改进条件。重复“评估 $\rightarrow$ 改进”会使策略价值单调不下降；有限 MDP 中策略数量有限，所以最终会达到稳定策略。若策略稳定，则它已经满足 Bellman 最优化方程，对应最优策略。

24.14 题 14：说明 off-policy MC 中重要性采样的作用，并比较普通重要性采样和加权重要性采样。

参考答案：

off-policy 学习中，行为策略 b 用来产生数据，目标策略 π 是想评估或优化的策略。由于数据分布来自 b 而不是 π ，需要用重要性采样修正分布差异。轨迹从 t 到终止前的权重为

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}.$$

目标策略价值可用 ρG_t 来估计。

普通重要性采样形式为

$$V(s) = \frac{1}{N} \sum_{i=1}^N \rho_i G_i.$$

它在理论上无偏，但方差可能非常大，尤其是长 episode 中多个概率比值连乘时容易爆炸。加权重要性采样为

$$V(s) = \frac{\sum_{i=1}^N \rho_i G_i}{\sum_{i=1}^N \rho_i}.$$

它通常有偏，但方差更小，实践中更稳定。简答时可总结为：重要性采样解决“用一个策略的数据评估另一个策略”的问题；普通重要性采样偏差小但方差大，加权重要性采样方差小但有一定偏差。

24.15 题 15：请解释 n -step 方法的意义，并说明它如何体现偏差和方差折中。

参考答案：

n -step 方法使用前 n 步真实奖励加上第 n 步之后的价值估计作为目标：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

对于动作价值控制， n -step SARSA 使用：

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q(S_{t+n}, A_{t+n}).$$

当 $n = 1$ 时，它退化为一部 TD，bootstrap 最强，方差较小但偏差较大；当 n 增大到 episode 结束时，它接近 MC，真实回报更多，偏差较小但方差较大。因此 n -step 方法在 TD 和 MC 之间建立桥梁。选择合适的 n 可以在学习速度、稳定性、偏差和方差之间折中。

24.16 题 16：什么是优势函数？为什么策略梯度、A-C、TRPO、PPO 中常用优势函数而不是直接用回报？

参考答案：

优势函数定义为

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

它表示在状态 s 下选择动作 a 相比该状态平均水平好多少。如果 $A^\pi(s, a) > 0$ ，说明该动作比平均动作好，应该提高其概率；若 $A^\pi(s, a) < 0$ ，则应该降低其概率。

策略梯度中有

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(A_t | S_t) A^\pi(S_t, A_t)].$$

使用优势函数相当于减去了 baseline $V^\pi(s)$ 。由于只依赖状态的 baseline 不改变梯度期望：

$$\mathbb{E}_{a \sim \pi}[\nabla_\theta \log \pi_\theta(a | s) b(s)] = 0,$$

所以优势函数不会引入系统性方向错误，却能降低方差。A-C 中常用 TD 误差

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

近似优势；TRPO 和 PPO 中也用 $\hat{A}_t$ 判断动作概率该增大还是减小。相比直接用 G_t ，优势函数更稳定、更适合深度策略优化。

24.17 题 17：DQN 为什么需要经验回放和目标网络？如果没有它们会出现什么问题？

参考答案：

DQN 用神经网络近似 $Q(s, a; \theta)$ ，目标为

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-).$$

与表格 Q-learning 相比，神经网络训练有两个突出不稳定因素：一是连续交互样本高度相关，不符合普通监督学习中近似独立同分布的假设；二是训练目标依赖网络自身，如果当前网络一更新，目标也跟着变化，会形成“追逐移动目标”的不稳定过程。

经验回放将转移样本 (s, a, r, s', d) 存入 replay buffer，并随机抽取 minibatch 训练。它可以打破样本相关性，提高数据复用率。目标网络使用较旧参数 θ^- 计算 TD target，每隔一段时间复制 $\theta^- \leftarrow \theta$ ，使目标在短时间内相对固定。

如果没有经验回放，训练样本相关性强，网络容易震荡或过拟合最近经验；如果没有目标网络，TD target 随在线网络同步剧烈变化，训练更容易发散。因此经验回放和目标网络是 DQN 稳定训练的两个关键技术。

24.18 题 18：比较价值函数方法和策略梯度方法。什么情况下更适合用策略梯度？

参考答案：

价值函数方法先学习 $V(s)$ 或 $Q(s, a)$ ，再由价值函数诱导策略，例如

$$\pi(s) = \arg \max_a Q(s, a).$$

代表算法有 DP、MC control、TD、SARSA、Q-learning 和 DQN。优点是样本效率较好，离散动作问题中实现直观；缺点是连续动作空间中 $\arg \max_a Q(s, a)$ 往往难以求解，而且难以直接表达随机策略。

策略梯度方法直接参数化策略 $\pi_\theta(a | s)$ ，并优化期望回报：

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta).$$

代表算法有 REINFORCE、Actor-Critic、TRPO、PPO。它适合连续动作控制、需要随机策略的问题，以及动作空间很大、难以对所有动作求最大值的问题。缺点是梯度估计方差较大、样本效率可能较低，训练稳定性需要 baseline、critic、trust region 或 PPO clip 等技巧。

总结：离散动作、可枚举动作时，价值法通常简单有效；连续动作、高维策略控制或需要直接优化随机策略时，策略梯度更自然。

24.19 题 19：解释 GAE 的思想。它与 TD、MC、优势估计有什么关系？

参考答案：

GAE (Generalized Advantage Estimation) 是一种优势估计方法，常用于 TRPO 和 PPO。先定义 TD 误差：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

GAE 将未来多个 TD 误差按指数衰减加权：

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}.$$

当 $\lambda = 0$ 时，GAE 主要使用一步 TD 误差，方差低但偏差较大；当 λ 接近 1 时，它包含更多未来奖励，更接近 MC 回报，偏差较小但方差较大。因此 GAE 是在 TD 和 MC 之间做偏差-方差折中的优势估计方法。

在策略优化中，优势函数告诉算法某个动作是否比该状态的平均水平更好。GAE 提供了更平滑、更稳定的 $\hat{A}_t$ ，因此能帮助 A-C、TRPO、PPO 获得更稳定的策略梯度。

24.20 题 20：请说明强化学习中的“探索与利用”问题，并分别举出价值法和策略法中的处理方式。

参考答案：

探索与利用是指智能体既要利用当前已知的高价值动作获取回报，又要探索未知动作以发现潜在更优策略。如果只利用，可能陷入局部最优；如果过度探索，则会牺牲当前性能并导致学习效率低。

在价值法中，常用 ϵ -greedy：

$$a = \begin{cases} \arg \max_a Q(s, a), & \text{概率 } 1 - \epsilon, \\ \text{随机动作}, & \text{概率 } \epsilon. \end{cases}$$

它简单地在贪心动作和随机动作之间平衡。也可以使用 softmax 策略，根据动作价值大小分配概率。

在策略梯度方法中，策略本身通常是随机策略，例如 categorical 或 Gaussian policy；训练中还常加入熵奖励：

$$S[\pi](s) = - \sum_a \pi(a | s) \log \pi(a | s),$$

鼓励策略保持一定随机性。PPO 等算法常在目标中加入 entropy bonus 防止策略过早坍塌。简言之，价值法常通过行为策略加入探索，策略法常通过随机策略和熵正则实现探索。

24.21 题 21：如果考试要求你为一个任务选择算法，应如何根据任务特点选择 DP、DQN 或 PPO？

参考答案：

若环境模型已知、状态动作空间较小，可以选择 DP，例如策略迭代或价值迭代。原因是 DP 能直接利用 $p(s', r | s, a)$ 计算 Bellman backup，理论清晰且不需要采样。但如果状态空间很大或模型未知，DP 不现实。

若模型未知、动作空间离散、状态可能是高维输入，可以选择 DQN。DQN 适合离散动作控制，通过神经网络近似 $Q(s, a)$ ，并用经验回放和目标网络稳定训练。典型场景是离散动作游戏或控制任务。若担心过估计，可以使用 Double DQN；若状态价值和动作优势需要分开表达，可使用 Dueling DQN。

若动作空间连续，或者希望直接学习随机策略，可以选择 PPO。PPO 属于策略梯度方法，使用 clipped surrogate 控制新旧策略差异，实现简单且稳定性较好，广泛用于连续控制和复杂智能体训练。简答时可概括：小规模已知模型选 DP；未知模型离散动作选 DQN；连续动作或直接策略优化选 PPO。

24.22 题 22：结合强化学习课程，谈谈未来强化学习研究可能面临的挑战和发展方向。

参考答案：

强化学习未来的重要挑战之一是样本效率。许多 RL 算法需要大量交互数据，而真实机器人、自动驾驶或医疗决策等场景中交互成本高、风险大。因此未来研究可能更多结合模型学习、离线强化学习和仿真到现实迁移，用已有数据或 learned world model 减少真实试错。

第二个挑战是训练稳定性和安全性。DQN、TRPO、PPO 等算法的发展说明，深度强化学习很容易受到目标非平稳、函数逼近误差和策略更新过大的影响。真实系统还要求智能体不能通过危险探索造成损失，因此安全约束、风险敏感优化、可解释策略和人类反馈会越来越重要。

第三个方向是与大模型和具身智能结合。大模型擅长知识理解和任务规划，但要真正成为智能体，还需要通过环境反馈学习长期决策。强化学习可以为工具调用、机器人操作、多智能体协作和自动科学发现提供决策优化框架。我的理解是，未来 RL 不会只作为单独算法存在，而会和监督学习、生成模型、世界模型、人类反馈以及安全机制结合，形成更可靠、更通用的智能体学习范式。

24.23 题 23：证明或说明 Bellman 最优算子为什么是收缩映射，并解释这对价值迭代收敛性的意义。

参考答案：

Bellman 最优算子定义为

$$(T_* V)(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

对任意两个价值函数 U, V ，有

$$\begin{aligned} |(T_*U)(s) - (T_*V)(s)| &= \left| \max_a f_a(U) - \max_a f_a(V) \right| \\ &\leq \max_a |f_a(U) - f_a(V)| \\ &= \max_a \left| \sum_{s',r} p(s',r | s, a) \gamma [U(s') - V(s')] \right| \\ &\leq \gamma \max_a \sum_{s',r} p(s',r | s, a) |U(s') - V(s')| \\ &\leq \gamma \|U - V\|_\infty. \end{aligned}$$

因此

$$\|T_*U - T_*V\|_\infty \leq \gamma \|U - V\|_\infty.$$

当 $\gamma < 1$ 时， T_* 是 γ -收缩映射。根据压缩映射定理，它有唯一不动点 V_* ，且反复迭代

$$V_{k+1} = T_*V_k$$

会收敛到 V_* 。这就是价值迭代能够收敛到最优价值函数的理论基础。考试中可进一步说明：策略评估中的 T_π 也是收缩映射，因此迭代策略评估会收敛到 v_π 。

24.24 题 24：深度强化学习中的 deadly triad 是什么？请结合 Q-learning、DQN 和 Double DQN 说明其影响与缓解方法。

参考答案：

deadly triad 指函数逼近、bootstrap 和 off-policy 三者同时出现时，强化学习训练容易不稳定甚至发散。函数逼近指用神经网络等近似 $Q(s, a)$ ；bootstrap 指目标中使用当前估计值，如

$$R + \gamma \max_{a'} Q(s', a');$$

off-policy 指行为策略和目标策略不同，如 Q-learning 用探索策略采样，却学习贪心目标策略。

表格 Q-learning 在一定条件下可以收敛，但 DQN 同时具有三者：神经网络函数逼近、TD bootstrap、Q-learning 的 off-policy 目标。因此它容易受到样本相关性、目标非平稳和过估计的影响。DQN 的经验回放缓解样本相关性，目标网络缓解目标非平稳：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-).$$

Double DQN 进一步缓解最大化带来的过估计，把动作选择和评估分开：

$$a^* = \arg \max_{a'} Q(s', a'; \theta),$$

$$y = r + \gamma Q(s', a^*; \theta^-).$$

因此，deadly triad 解释了深度价值学习为什么难以稳定；DQN 系列的关键改进都可以看成是在缓解三者叠加造成的不稳定性。

24.25 题 25：解释 Q-learning 和 DQN 中的过估计偏差。Double DQN 为什么能缓解这一问题？

参考答案：

Q-learning 的目标包含最大化操作：

$$R + \gamma \max_{a'} Q(S', a').$$

若 $Q(S', a')$ 是带噪声的估计，最大化操作倾向于选择被噪声高估的动作。即使每个动作价值估计本身是无偏的，最大值的期望也可能偏大：

$$\mathbb{E}[\max_a \hat{Q}(a)] \geq \max_a \mathbb{E}[\hat{Q}(a)].$$

这就是过估计偏差。DQN 中神经网络估计误差更明显，因此过估计可能导致目标值偏高，进而影响策略选择和训练稳定性。

Double DQN 的核心思想是把“选动作”和“评估动作”分离。普通 DQN 用同一个目标网络完成最大化和估值：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-).$$

Double DQN 用在线网络选动作：

$$a^* = \arg \max_{a'} Q(s', a'; \theta),$$

再用目标网络评估该动作：

$$y = r + \gamma Q(s', a^*; \theta^-).$$

由于选择和评估不完全使用同一组误差，最大化噪声被放大的程度降低，因此过估计偏差减轻。它不是完全消除误差，而是减少“同一估计器既挑最大又给分”带来的系统性高估。

24.26 题 26：推导策略梯度定理的核心步骤，并说明为什么 baseline 不改变梯度期望。

参考答案：

设轨迹为 $\tau = (s_0, a_0, r_1, \dots)$ ，目标为

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)].$$

轨迹概率为

$$p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t | s_t) p(s_{t+1}, r_{t+1} | s_t, a_t).$$

环境转移不依赖 θ ，因此

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t).$$

利用 log-derivative trick：

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau),$$

得到

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(A_t | S_t) G_t \right].$$

这就是 REINFORCE 的基础。

baseline 不改变梯度期望，因为对任意只依赖状态的 $b(s)$ ：

$$\begin{aligned} \mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) b(s)] &= b(s) \sum_a \pi_\theta(a | s) \frac{\nabla_\theta \pi_\theta(a | s)}{\pi_\theta(a | s)} \\ &= b(s) \nabla_\theta \sum_a \pi_\theta(a | s) \\ &= b(s) \nabla_\theta 1 = 0. \end{aligned}$$

因此可以用 $G_t - b(S_t)$ 或优势函数 $A^\pi(S_t, A_t)$ 替代 G_t ，不改变期望梯度方向，但能显著降低方差。这也是 Actor-Critic 使用 critic 的理论基础。

24.27 题 27: TRPO 和 PPO 都限制新旧策略差异。请从优化目标、约束方式和近似误差角度深入比较二者。

参考答案:

TRPO 和 PPO 的共同目标是: 在利用策略梯度提升性能的同时, 避免新策略离旧策略太远。TRPO 显式求解约束优化:

$$\begin{aligned} & \max_{\theta} \mathbb{E}_t[r_t(\theta) \hat{A}_t], \\ \text{s.t. } & \mathbb{E}_t[D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta})] \leq \delta. \end{aligned}$$

它将目标一阶近似、KL 二阶近似, 得到自然梯度方向:

$$x^* \propto H^{-1}g.$$

实际中用共轭梯度求 $H^{-1}g$, 再用线搜索保证 KL 约束和性能改进。TRPO 的优点是理论动机强、更新保守稳定; 缺点是实现复杂、计算成本高。

PPO 不直接解 KL 约束优化, 而是使用裁剪目标:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)].$$

当 $\hat{A}_t > 0$ 时, 若 $r_t > 1 + \epsilon$, 继续增大该动作概率不会带来更多收益; 当 $\hat{A}_t < 0$ 时, 若 $r_t < 1 - \epsilon$, 继续降低概率也不会带来更多收益。这样 PPO 用简单的下界式目标近似 trust region, 避免过大更新。

更深入地说, TRPO 用平均 KL 显式约束策略分布整体变化, PPO 用样本级概率比率裁剪间接限制变化。PPO 不保证严格满足 KL 约束, 因此理论保证弱于 TRPO; 但它可用普通 Adam/minibatch 多轮训练, 工程上更简单、效率更高。考试中可总结: TRPO 是“理论更干净的信赖域约束”, PPO 是“工程更实用的裁剪近似”。

24.28 题 28: 请从“目标值 target”的角度统一比较 MC、TD、SARSA、Q-learning 和 DQN。

参考答案:

这些算法都可以写成统一形式:

$$\text{估计值} \leftarrow \text{估计值} + \alpha(\text{target} - \text{估计值}).$$

区别主要在 target 的构造方式。MC 使用完整真实回报:

$$\text{target}_{\text{MC}} = G_t.$$

TD(0) 使用一步 bootstrap:

$$\text{target}_{\text{TD}} = R_{t+1} + \gamma V(S_{t+1}).$$

SARSA 用实际下一动作:

$$\text{target}_{\text{SARSA}} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}),$$

因此是同策略。Q-learning 用下一状态最大动作价值:

$$\text{target}_{\text{Q}} = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'),$$

因此是异策略最优控制。DQN 将表格 Q 替换为神经网络 $Q(s, a; \theta)$, 并使用目标网络计算 target:

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-).$$

所以这些算法的递进可以概括为: MC 用完整回报, TD 用一步价值估计, SARSA/Q-learning 把预测推广到控制, DQN 把表格动作价值函数推广为深度函数逼近。

24.29 题 29：为什么 Q-learning 是异策略算法但通常不需要像 off-policy MC 那样显式使用重要性采样？

参考答案：

off-policy MC 用行为策略 b 采样、评估目标策略 π 的完整回报。由于整条轨迹分布不同，需要重要性采样修正：

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}.$$

否则用 b 下的回报均值估计 π 的价值会有分布偏差。

Q-learning 也是异策略，因为行为策略可以是 ϵ -greedy，而目标策略是贪心策略。但 Q-learning 不直接估计某条目标策略轨迹下的完整回报，而是对 Bellman 最优方程做随机近似：

$$q_*(s, a) = \mathbb{E}[R + \gamma \max_{a'} q_*(S', a') | s, a].$$

只要行为策略能充分探索所有状态动作对，样本 (S, A, R, S') 就可用于更新最优 Bellman target：

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_{a'} Q(S', a') - Q(S, A)].$$

因此 Q-learning 的 off-policy 性来自 target 中的 max，而不是通过重要性采样修正整条轨迹分布。简言之：off-policy MC 是“用一条行为轨迹估计目标策略回报”，需要 IS；Q-learning 是“用样本转移逼近 Bellman 最优算子”，通常不显式需要 IS。

24.30 题 30：Dueling DQN 为什么要写成 $Q = V + A - \bar{A}$ ？如果直接写 $Q = V + A$ 会有什么问题？

参考答案：

Dueling DQN 将动作价值函数拆成状态价值和优势函数：

$$Q(s, a) = V(s) + A(s, a).$$

直观上， $V(s)$ 表示状态本身好不好， $A(s, a)$ 表示动作相对该状态平均水平好不好。但直接写 $Q = V + A$ 会产生不可辨识性：对任意常数 c ，令

$$V'(s) = V(s) + c, \quad A'(s, a) = A(s, a) - c,$$

则

$$V'(s) + A'(s, a) = V(s) + A(s, a) = Q(s, a).$$

也就是说，同一个 Q 可以由无数组 V, A 表示，训练时网络分支含义不稳定。

因此 Dueling DQN 通常使用聚合形式：

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right).$$

减去平均优势后，优势函数在动作维度上均值为零，使 $V(s)$ 更接近该状态的平均价值， $A(s, a)$ 更接近动作相对优势。这样既解决不可辨识性，又能在许多动作价值差异不明显的状态中更有效地学习状态价值。

24.31 题 31：请比较普通 DQN、Double DQN、Dueling DQN 三者分别解决了什么问题，是否可以组合使用？

参考答案：

普通 DQN 解决的是表格 Q-learning 无法处理高维状态的问题。它用神经网络近似 $Q(s, a; \theta)$ ，并使用经验回放和目标网络稳定训练：

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-).$$

Double DQN 解决的是最大化操作带来的过估计问题。它用在线网络选动作、目标网络评估动作：

$$a^* = \arg \max_{a'} Q(s', a'; \theta),$$

$$y = r + \gamma(1 - d) Q(s', a^*; \theta^-).$$

Dueling DQN 解决的是网络表达结构问题。它将 Q 分为 $V(s)$ 和 $A(s, a)$ ，使网络更容易学习状态价值和动作差异：

$$Q(s, a) = V(s) + A(s, a) - \bar{A}(s).$$

三者解决的问题不同：DQN 是基础深度价值学习框架；Double DQN 改 target，降低过估计；Dueling DQN 改网络结构，提高表达效率。因此它们可以组合，例如 Dueling Double DQN，即使用 dueling 网络结构，同时用 Double DQN 的 target 训练。

24.32 题 32：Actor-Critic 中 critic 估计不准会对 actor 产生什么影响？如何缓解？

参考答案：

Actor-Critic 用 critic 给 actor 提供更新信号。一步 A-C 中，critic 估计 $V_w(s)$ ，并用 TD 误差

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)$$

近似优势函数，actor 更新为

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

如果 critic 估计不准， δ_t 的符号和大小可能错误，actor 会提高坏动作概率或降低好动作概率，从而导致策略学习不稳定甚至退化。

缓解方法包括：第一，给 critic 足够训练，使价值损失

$$(G_t^{(n)} - V_w(S_t))^2$$

或 TD loss 下降；第二，使用 n -step return 或 GAE，在偏差和方差之间取得更好折中；第三，对 advantage 做标准化，减少尺度不稳定；第四，使用较小 actor 学习率，避免 critic 尚未准确时策略变化过大；第五，在 PPO/TRPO 中限制新旧策略差异，降低错误优势估计造成的破坏。简言之，A-C 的关键难点是 actor 和 critic 相互依赖：critic 降低方差，但 critic 偏差也会传递给 actor。

24.33 题 33：为什么 PPO 可以对同一批数据训练多个 epoch，而普通 on-policy 策略梯度通常不适合无限重复使用旧数据？

参考答案：

on-policy 策略梯度要求数据来自当前策略 π_θ 。当策略更新后，旧数据其实来自旧策略 $\pi_{\theta_{\text{old}}}$ ，若继续当成当前策略数据使用，会产生分布偏差。PPO 使用重要性比率修正新旧策略差异：

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

同时使用裁剪目标：

$$L^{\text{CLIP}} = \mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)].$$

当新策略相对旧策略变化过大时，裁剪会限制目标继续鼓励这种变化。

因此 PPO 可以在同一批 rollout 上进行多个 minibatch epoch，提高样本利用率。但这种重复使用不是无限制的：如果训练太多轮，新策略和采样旧策略差异太大，重要性比率会偏离 1，裁剪后有效学习信号变弱，甚至产生偏差。因此 PPO 的多 epoch 复用依赖 clip 或 KL early stopping，本质上仍是近似 on-policy，而不是完全 off-policy。

24.34 题 34：请解释“奖励稀疏”和“延迟奖励”为什么会使强化学习困难，并说明 MC、TD、n-step、GAE 如何处理信用分配问题。

参考答案：

奖励稀疏指大多数时间步奖励为零，只有少数关键状态有奖励；延迟奖励指动作的好坏要经过很长时间才体现。这会带来信用分配困难：智能体很难判断早期哪些动作导致了最终奖励。

MC 使用完整回报 G_t ，能把终局奖励传播给整条 episode 中的早期状态动作，因此理论上适合处理延迟奖励；但它必须等 episode 结束，且方差大。TD 使用一步目标

$$R + \gamma V(S')$$

逐步传播价值，方差较小、可在线更新，但远期奖励传播较慢。n-step 方法使用

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}),$$

比一步 TD 更快传播远期奖励，又比 MC 方差小。GAE 将多个 TD 误差加权求和：

$$\hat{A}_t = \sum_l (\gamma \lambda)^l \delta_{t+l},$$

在策略梯度中更稳定地估计优势。

因此，MC、TD、n-step 和 GAE 可以看成处理信用分配的不同方式：MC 长视野但高方差；TD 短视野但低方差；n-step 和 GAE 在二者之间折中。

24.35 题 35：如果环境模型是错的，基于模型方法会出现什么问题？Dyna-Q 如何在样本效率和模型偏差之间折中？

参考答案：

基于模型方法依赖模型预测转移和奖励。如果模型错误，则规划时会使用错误的模拟经验，导致价值函数向错误方向更新。尤其在 Dyna-Q 中，模拟更新为

$$Q(\tilde{S}, \tilde{A}) \leftarrow Q(\tilde{S}, \tilde{A}) + \alpha [\tilde{R} + \gamma \max_{a'} Q(\tilde{S}', a') - Q(\tilde{S}, \tilde{A})],$$

若 $(\tilde{R}, \tilde{S}')$ 来自错误模型，则错误会被多次规划更新放大。

Dyna-Q 的优势是样本效率高：一次真实交互不仅更新 Q ，还更新模型，并可产生多次模拟更新。这在真实交互昂贵时很有价值。但规划步数越多，对模型准确性的依赖越强；模型越不准，偏差传播越严重。因此 Dyna-Q 在样本效率和模型偏差之间折中：规划步数少，样本利用不足但更稳；规划步数多，学习快但更依赖模型。

若环境会变化，还需要及时更新模型，甚至增加探索或使用 Dyna-Q+ 之类方法鼓励访问长期未尝试的动作。简答时可总结：model-based 方法的核心风险是“模型错，规划也错”；Dyna-Q 的核心思想是用少量真实经验加大量模拟经验提高效率，但必须控制模型误差。

24.36 题 36：请从偏差-方差角度比较 MC、TD、REINFORCE、Actor-Critic、TRPO/PPO。

参考答案：

MC 使用完整回报，目标更接近真实回报，因此偏差低，但不同 episode 回报波动大，方差高。TD 使用 bootstrap：

$$R + \gamma V(S'),$$

由于依赖当前价值估计，会引入偏差，但方差较低，能在线更新。

REINFORCE 是策略梯度中的 MC 方法，用 $G_t \nabla \log \pi_\theta$ 更新。它梯度估计通常无偏，但方差很高。Actor-Critic 引入 critic，用 TD 误差或优势函数替代完整回报：

$$\delta_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

这样显著降低方差，但 critic 近似会引入偏差。

TRPO/PPO 通常建立在 Actor-Critic 和优势估计之上。它们不只是处理偏差方差，还额外处理策略更新稳定性。GAE 中 λ 控制偏差-方差： λ 小更像 TD，方差低但偏差高； λ 大更像 MC，偏差低但方差高。PPO 的 clip 和 TRPO 的 KL 约束则限制策略更新幅度，降低高方差梯度造成的性能崩溃风险。整体来看，强化学习算法常用一定偏差换取更低方差和更稳定训练。

24.37 题 37：请设计一个答题框架：若老师要求“比较两个强化学习算法”，应该从哪些维度回答？

参考答案：

比较两个强化学习算法时，可以按固定框架作答。第一，说明二者解决的问题和所属类别，例如有无模型、同策略/异策略、基于价值/基于策略、在线/离线。第二，写核心公式，明确 target 或目标函数。例如 SARSA 写

$$R + \gamma Q(S', A'),$$

Q-learning 写

$$R + \gamma \max_{a'} Q(S', a').$$

第三，比较更新流程，包括是否需要完整 episode、是否 bootstrap、是否需要模型、是否需要 replay buffer 或目标网络。

第四，比较偏差方差和稳定性。例如 MC 无 bootstrap、方差大；TD bootstrap、方差小但有偏；TRPO/PPO 通过限制策略变化提高稳定性。第五，比较适用场景。例如 DP 适合已知模型小规模问题，DQN 适合离散动

作高维状态，PPO 适合连续动作和直接策略优化。第六，写递进关系，即后一个算法解决前一个算法的什么问题。

因此开卷考试中遇到比较题，可以按“类别 → 公式 → 流程 → 优缺点 → 适用场景 → 递进关系”展开，基本能覆盖大部分得分点。